

IF2224 TEORI BAHASA FORMAL DAN OTOMATA
LAPORAN TUGAS BESAR
MILESTONE 2



Disusun oleh:

Kelompok 01 – Kelas 01

Muhammad Jordan Ferimeison (13524047)

Arina Azka (13524049)

Hakam Avicena Mustain (13524075)

Yavie Azka Putra Araly (13524077)

Angelina Andra Alanna (13524079)

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

BAB I

LANDASAN TEORI

1.1. Parser

Parser atau syntax analyzer adalah komponen dalam compiler yang bertugas memeriksa apakah urutan token yang dihasilkan oleh lexer sesuai dengan aturan tata bahasa (grammar) dari bahasa pemrograman yang dikompilasi. Jika lexer memecah source code menjadi token-token atomik, maka parser mengambil aliran token tersebut dan membangun parse tree yang merepresentasikan struktur sintaks program.

Secara umum, parser memiliki beberapa tugas.

1. Menerima aliran token dari lexer sebagai input.
2. Menentukan apakah aliran token tersebut merupakan kalimat yang valid secara sintaksis berdasarkan grammar bahasa kode sumber.
3. Membangun parse tree sebagai model sintaksis yang menjadi masukan bagi tahap semantic analysis.
4. Mendeteksi dan melaporkan kesalahan (syntax error) dengan pesan yang informatif.

Model sintaksis yang dihasilkan oleh parser diekspresikan sebagai grammar formal G . Untuk suatu aliran token s dan grammar G , parser berusaha membangun bukti konstruktif bahwa s dapat diturunkan dalam G . Proses inilah yang disebut parsing.

Algoritma parsing secara umum terbagi menjadi dua kategori. Sementara parser top-down dimulai dari simbol awal dan bergerak ke bawah menuju terminal, parser bottom-up dimulai dari terminal dan bergerak ke atas menuju simbol awal.

Untuk mendeskripsikan sintaks bahasa pemrograman, parser menggunakan Context-Free Grammar (CFG). Regular expression yang digunakan pada tahap lexer tidak memiliki kemampuan yang cukup untuk mendeskripsikan sintaksis penuh dari sebagian besar bahasa pemrograman. Maka dari itu, CFG diperlukan sebagai notasi yang lebih ekspresif untuk mendeskripsikan sintaksis sekaligus menyediakan mekanisme untuk memeriksa keanggotaan suatu string dalam bahasa yang didefinisikan oleh grammar tersebut.

Parser beroperasi pada Context-Free Grammar (CFG). CFG didefinisikan sebagai tuple $G = (V, \Sigma, R, S)$ di mana:

- V = himpunan variabel/nonterminal (mis. $\langle \text{program} \rangle$, $\langle \text{expression} \rangle$)
- Σ = himpunan terminal/token (mis. programsy , ident , intcon)
- R = himpunan aturan produksi, masing-masing berbentuk $A \rightarrow \alpha$
- S = simbol awal (start symbol), yaitu $\langle \text{program} \rangle$

Tujuan parser dalam kompilasi Arion Compiler adalah:

- Memastikan urutan token membentuk struktur sintaks yang valid berdasarkan grammar bahasa Arion.
- Mendeteksi dan melaporkan kesalahan (syntax error) dengan pesan yang informatif.
- Membangun Parse Tree yang menjadi masukan bagi tahap semantic analysis.

1.2. Context-Free Grammars

Context-Free Grammar (CFG). CFG adalah sekumpulan aturan yang mendeskripsikan bagaimana membentuk kalimat-kalimat yang valid dalam suatu bahasa. Kumpulan kalimat yang dapat diturunkan dari grammar G disebut bahasa yang didefinisikan oleh G , dinotasikan sebagai $L(G)$. CFG didefinisikan sebagai tuple $G = (V, \Sigma, R, S)$ di mana:

- V = himpunan variabel/nonterminal (mis. $\langle \text{program} \rangle$, $\langle \text{expression} \rangle$)
- Σ = himpunan terminal/token (mis. programsy , ident , intcon)
- R = himpunan aturan produksi, masing-masing berbentuk $A \rightarrow \alpha$
- S = simbol awal (start symbol), yaitu $\langle \text{program} \rangle$

Untuk menurunkan sebuah kalimat, proses dimulai dari string awal yang hanya berisi start symbol. Kemudian dipilih satu simbol nonterminal dalam string tersebut, dipilih satu aturan produksi yang sesuai, dan nonterminal tersebut ditulis ulang dengan sisi kanan produksi. Proses penulisan ulang ini diulangi hingga string tidak lagi mengandung simbol nonterminal dan seluruhnya terdiri dari simbol terminal. Pada titik inilah string tersebut menjadi sebuah kalimat yang valid dalam bahasa.

Pada setiap langkah dalam proses penurunan, string yang terbentuk merupakan campuran simbol terminal dan nonterminal. String semacam ini disebut sentential form. Setiap sentential form dapat diturunkan dari start symbol dalam nol atau lebih langkah,

dan dari setiap sentential form dapat diturunkan kalimat yang valid dalam nol atau lebih langkah pula.

Notasi tradisional yang digunakan untuk merepresentasikan CFG disebut Backus-Naur Form (BNF). BNF memiliki 4 simbol dasar berikut.

Simbol	Nama	Makna
::=	Didefinisikan sebagai	nonterminal di kiri diturunkan menjadi simbol di kanan
	Atau	Pemisah alternatif produksi
<>	Kurung sudut	Mengapit simbol nonterminal
Teks biasa	Terminal	Simbol yang tidak diapit < > adalah terminal

Sebagai contoh, produksi untuk <constant> dalam bahasa Pascal adalah sebagai berikut.

<pre> <constant> ::= intcon realcon ident + intcon - intcon </pre>
--

Untuk mengekspresikan produksi secara lebih ringkas, digunakan EBNF (Extended Backus-Naur Form) yang merupakan perluasan dari BNF. EBNF mewarisi semua simbol BNF dan menambahkan tiga simbol baru.

Simbol	Nama	Makna
{ x }	Kurung kurawal	x diulang nol atau lebih kali
[x]	Kurung siku	x bersifat opsional (nol atau satu kali)
(a b)	Kurung biasa	Pengelompokan pilihan

```
<const-declaration> ::= constsy { ident = <constant> ; }  
<procedure-declaration> ::= proceduresy ident  
                                [ <formal-parameter-list> ] ;  
                                <block> ;
```

1.3. Parse Tree

Parse tree adalah struktur pohon yang merepresentasikan struktur sintaksis dari suatu string berdasarkan grammar formal, biasanya context-free grammar. Dalam desain compiler, parse tree menunjukkan bagaimana suatu potongan kode atau kalimat input dapat diturunkan dari aturan grammar bahasa tersebut. Root dari pohon adalah simbol awal, node internal adalah simbol nonterminal, dan node daun adalah simbol terminal atau token. Karena struktur ini, parse tree membuat organisasi gramatikal dari program menjadi terlihat dan lebih mudah dianalisis.

Parse tree berkaitan dengan proses parsing, yaitu tahap kompilasi yang menentukan apakah urutan token sesuai dengan grammar bahasa. Parsing memeriksa kebenaran sintaks dan membangun representasi struktural dari input. Representasi ini berguna karena memperlihatkan hubungan sintaksis antarbagian program, seperti ekspresi, statement, dan deklarasi. Dengan kata lain, parse tree adalah hasil penerapan aturan grammar secara bertahap sampai string input dikenali.

1.3.1. Definisi dan Struktur

Parse tree memiliki beberapa sifat struktural penting:

- Root adalah simbol awal grammar
- Setiap node internal merepresentasikan simbol nonterminal.
- Setiap node daun merepresentasikan simbol terminal, token, atau unit leksikal.
- Anak dari suatu node sesuai dengan simbol-simbol pada sisi kanan produksi grammar.
- Jika daun dibaca dari kiri ke kanan, hasilnya adalah string input asli.

Struktur ini menunjukkan bagaimana sebuah string input diturunkan dari grammar. Setiap parse tree merepresentasikan satu proses derivasi, dan setiap derivasi yang valid dapat digambarkan dalam bentuk parse tree.

1.3.2. Peran dalam Desain Compiler

Parse tree berperan sebagai penghubung antara fase analisis leksikal dan analisis sintaks dalam proses kompilasi, Lexical analyzer mengubah source code menjadi urutan token, kemudian parser memproses token tersebut untuk memverifikasi kesesuaian grammar. Jika valid, parser akan membangun parse tree yang merepresentasikan struktur gramatikal program.

Parse tree digunakan dalam tahap-tahap lanjutan kompilasi, yaitu:

- Semantic Analysis: Tahap memeriksa tipe data, deklarasi variabel, dan aturan scope
- Code generation: Tahap untuk membantu pembentukan kode berdasarkan struktur, nesting, dan precedence operasi.

1.3.3. Parse Tree dan Grammar

Parse tree dibangun berdasarkan aturan grammar. Grammar terdiri dari simbol terminal, simbol nonterminal, simbol awal, dan aturan produksi. Saat parser membangun parse tree, parser menerapkan aturan produksi untuk memperluas simbol non terminal hingga hanya tersisa simbol terminal pada daun. Dengan demikian, parse tree merupakan representasi visual dari proses derivasi grammar. Sebagai contoh, misalkan grammar sederhana untuk ekspresi aritmatika:

- $E \rightarrow E + T \mid T$
- $T \rightarrow id$

Untuk input:

- $id + id + id$

<i>Tabel 1.3.3 Parse Tree yang dihasilkan</i>
E /\ E + T / E + T id T id id

visualisasi ini menunjukkan bahwa ekspresi diparsing sebagai:

- (id + id) + id

Hal ini memperlihatkan bagaimana parse tree membantu memahami:

- Precedence operator (prioritas operasi)
- Associativity (arah pengelompokan, kiri/kanan)

Struktur pohon menunjukkan operasi mana yang dievaluasi terlebih dahulu dan bagaimana ekspresi ditafsirkan secara sintaksis.

1.3.4. Parse Tree dan Abstract Syntax Tree

Parse tree merepresentasikan seluruh struktural gramatikal hasil derivasi, termasuk semua simbol yang digunakan dalam grammar. Sedangkan, abstract syntax tree (AST) hanya menyimpan struktur inti dan menghilangkan elemen gramatikal yang tidak diperlukan.

Sehingga, parse tree lebih mendetail dan berukuran lebih besar, sedangkan AST lebih ringkas dan efisien. Dalam implementasi compiler, parse tree biasanya digunakan sebagai tahap antara untuk membentuk AST. AST kemudian digunakan pada tahap lanjutan seperti semantic analysis dan optimasi.

Tabel 1.3 Perbedaan antara Parse Tree dan AST (Abstract Syntax Tree)	
Input	Grammar
5 + 3 * 2	$\text{expr} \rightarrow \text{expr} + \text{term} \mid \text{term}$ $\text{term} \rightarrow \text{term} * \text{factor} \mid \text{factor}$ $\text{factor} \rightarrow \text{number}$
Abstract Syntax Tree	Parse Tree
<pre> (+) / \ 5 (*) / \ 3 2 </pre>	<pre> <expr> ├── <expr> │ ├── <term> │ │ ├── <factor> │ │ │ └── number: 5 │ └── '+' │ └── <term> │ ├── <term> │ │ ├── <factor> │ │ │ └── number: 3 │ └── '*' │ └── <factor> │ └── number: 2 </pre>

1.4. Top-Down Parsing

Top-down parsing adalah suatu metode syntax analysis yang dimulai dari simbol awal (akar dari parse tree) dan memperluas pohon ke bawah langkah demi langkah hingga menemukan daun yang cocok dengan terminal, yaitu simbol akhir dari Grammar yang tidak dapat diturunkan lagi. Pada setiap titik, proses mempertimbangkan pohon parse yang sebagian telah dibangun. Parser memilih simbol nonterminal, yaitu simbol yang dapat diturunkan untuk menjadi simbol terminal, pada tepi bawah pohon dan

memperluasnya dengan menambahkan anak-anak sesuai dengan sisi kanan produksi dari nonterminal tersebut.

Proses ini terus berulang hingga salah satu kondisi berikut terpenuhi.

- Seluruh tepi bawah pohon parse hanya berisi simbol terminal dan aliran token input telah habis seluruhnya, yang berarti parsing berhasil.
- Terjadi ketidakcocokan antara simbol pada tepi bawah pohon parse dengan token yang ada pada aliran input, yang berarti terdapat kesalahan sintaks pada program yang sedang dianalisis.

Algoritma top-down parser bekerja dengan membangun leftmost derivation, yaitu selalu memperluas nonterminal paling kiri terlebih dahulu. Parser menggunakan sebuah stack untuk melacak bagian tepi bawah pohon yang belum dicocokkan. Pada setiap iterasi, parser berfokus pada simbol paling kiri yang belum dicocokkan. Jika simbol tersebut adalah nonterminal, parser memilih suatu produksi, membangun bagian pohon yang sesuai, lalu memindahkan fokus ke simbol paling kiri dari bagian baru tersebut. Jika simbol tersebut adalah terminal, parser mencocokkannya dengan token berikutnya dari aliran input. Jika cocok, fokus maju ke simbol berikutnya pada tepi bawah dan aliran input pun maju. Jika tidak cocok, parser harus melakukan backtracking.

Proses backtracking dilakukan dengan cara mengembalikan fokus ke induk dari node saat ini pada pohon parse yang sedang dibangun, lalu memutus hubungan dengan anak-anaknya. Jika masih ada produksi lain yang belum dicoba untuk nonterminal tersebut, parser memperluas kembali node tersebut menggunakan produksi berikutnya. Jika seluruh produksi yang tersedia sudah dicoba dan semuanya gagal, parser naik satu level lebih tinggi dan mencoba ulang dari sana. Ketika semua kemungkinan telah habis tanpa menemukan kecocokan, parser melaporkan syntax error dan berhenti. Selain itu, setiap kali backtracking dilakukan, parser juga harus mengembalikan token-token yang sudah dikonsumsi ke dalam aliran input, karena token-token tersebut mungkin dibutuhkan oleh produksi lain yang akan dicoba. Meskipun backtracking menjamin parser dapat mencoba semua kemungkinan, cara ini memiliki biaya komputasi yang tinggi sehingga dalam praktiknya sedapat mungkin dihindari.

Salah satu hal yang membuat top-down parsing menjadi efisien adalah bahwa sebagian besar context-free grammar dapat di-parse tanpa backtracking. Hal ini

dimungkinkan melalui transformasi grammar tertentu sehingga parser dapat selalu memilih produksi yang tepat hanya berdasarkan token yang sedang dilihat, tanpa perlu mundur. Dua teknik yang umum digunakan untuk membangun top-down parser tanpa backtracking adalah recursive descent parser yang ditulis secara manual, dan LL(1) parser yang dibangkitkan secara otomatis.

1.5. Recursive Descent Parsing

Recursive descent parsing adalah teknik membangun parser top-down yang bekerja tanpa backtracking. Parser ini disusun sebagai sekumpulan prosedur yang saling memanggil satu sama lain, artinya setiap prosedur bertanggung jawab untuk mengenali satu nonterminal dalam grammar. Karena strukturnya mengikuti grammar secara langsung, grammar itu sendiri menjadi panduan utama dalam implementasi parser.

Pada recursive-descent parsing, setiap nonterminal dalam grammar dipetakan menjadi tepat satu fungsi. Ketika suatu fungsi perlu mengenali nonterminal lain di sisi kanan produksinya, ia cukup memanggil fungsi yang bersesuaian. Sementara itu, untuk simbol terminal, pencocokan dilakukan secara langsung dengan token dari input stream.

1.6. Error Handling

Dalam praktiknya, programmer sering mengompilasi kode yang mengandung kesalahan sintaks. Compiler bahkan diterima secara luas sebagai cara tercepat untuk menemukan kesalahan semacam itu. Oleh karena itu, parser idealnya mampu menemukan sebanyak mungkin kesalahan sintaks dalam satu kali proses parsing.

Parser yang langsung berhenti ketika menemukan kesalahan pertama akan mencegah compiler membuang waktu menerjemahkan program yang salah, tetapi akibatnya compiler hanya dapat melaporkan satu kesalahan per kompilasi. Hal ini menyulitkan proses pencarian seluruh kesalahan dalam sebuah file. Untuk mengatasi hal tersebut, parser perlu memiliki mekanisme error recovery, yaitu kemampuan untuk pulih dari kesalahan dan melanjutkan proses parsing.

Salah satu cara yang umum digunakan adalah dengan memilih satu atau beberapa token sebagai synchronizing word. Ketika parser menemukan kesalahan, parser

membuang token-token dari input stream hingga menemukan synchronizing word, lalu mereset kondisi internalnya agar konsisten dengan token tersebut.

Pada bahasa pemrograman bergaya Algol yang menggunakan titik koma sebagai pemisah pernyataan, titik koma sering dijadikan synchronizing word. Ketika terjadi kesalahan, parser terus membuang token hingga menemukan titik koma, lalu melanjutkan parsing seolah-olah sebuah pernyataan telah berhasil dikenali.

Dalam recursive descent parser, mekanisme ini dapat diimplementasikan dengan membuang token hingga synchronizing word ditemukan, kemudian mengembalikan kendali ke titik di mana pernyataan dianggap berhasil diproses.

BAB II

PERANCANGAN DAN IMPLEMENTASI

2.1. Gambaran Umum

Parser pada Arion Compiler merupakan tahap kedua dalam pipeline kompilasi, yaitu syntax analysis. Parser menerima aliran token yang dihasilkan oleh lexer dari milestone 1 dan membangun parse tree yang merepresentasikan struktur sintaksis program Arion secara hierarkis. Pipeline secara keseluruhan adalah sebagai berikut.

Source code → Lexer → Token Stream → Parser → Parse Tree

Parser diimplementasikan menggunakan algoritma Recursive Descent dengan grammar bahasa Arion yang di-hardcode langsung dalam program. Input parser berupa file .txt hasil tokenisasi dari lexer, dan output-nya adalah parse tree yang dicetak ke terminal sekaligus disimpan ke file .txt.

2.2. Perancangan Grammar

Berikut ini grammar bahasa Arion yang dirancang dalam notasi EBNF.

```
<program> ::= <program-header>
            <declaration-part>
            <compound-statement>
            period

<program-header> ::= programsy
                  ident
                  semicolon

<declaration-part> ::= { <const-declaration> }
                    { <type-declaration> }
                    { <var-declaration> }
                    { <subprogram-declaration> }

<const-declaration> ::= constsy
```

```

        { ident
          eql
          <constant>
          semicolon }

<constant> ::= charcon
            | string
            | [ (plus | minus) ]
              ( ident | intcon | realcon )

<type-declaration> ::= "typesy" ( ident eql <type> semicolon )+

<var-declaration> ::= "varsy" ( <identifier-list> colon <type> semicolon
)+

<identifier-list> ::= ident { comma ident }

<type> ::= <array-type>
        | <enumerated>
        | <record-type>
        | <range>
        | <constant>
        | ident

<array-type> ::= arraysy lbrack ( range | ident ) rbrack ofsy type

<range> ::= <constant> period period <constant>

<enumerated> ::= lparent ident { comma ident }* rparent

<record-type> ::= recordsy <field-list> endsy

<field-list> ::= <field-part> { semicolon <field-part> }

<field-part> ::= <identifier-list> colon <type>

```



```

        | <statement> { semicolon <statement> }

<statement> ::= <assignment-statement>
              | <if-statement>
              | <case-statement>
              | <while-statement>
              | <repeat-statement>
              | <for-statement>
              | <procedure/function-call>
              | ε

<variable> ::= ident
            | <component-variable>

<component-variable> ::= <variable> { ( lbrack <index-list> rbrack )
                                   | ( period ident ) }

<index-list> ::= ( intcon | charcon | ident )
                { comma ( intcon | charcon | ident ) }

<assignment-statement> ::= <variable>
                           becomes
                           <expression>

<if-statement> ::= ifsy
                 <expression>
                 thensy
                 <statement>
                 [ elsesy <statement> ]

<case-statement> ::= casesy
                  <expression>
                  ofsy
                  <case-block>
                  { semicolon <case-block> }
                  endsy

```

```

<case-block> ::= <constant>
                { comma <constant> }
                colon
                <statement>

<while-statement> ::= whilesy
                    <expression>
                    dosy
                    <statement>

<repeat-statement> ::= repeatsy
                    <statement-list>
                    untilsy
                    <expression>

<for-statement> ::= forsy
                    ident
                    becomes
                    <expression>
                    ( tosy | downtosy )
                    <expression>
                    dosy
                    <statement>

<procedure/function-call> ::= <variable>
                               [ lparent [ <parameter-list> ] rparent ]

<parameter-list> ::= <expression>
                    { comma <expression> }

<expression> ::= <simple-expression>
                [ ( <relational-operator> <simple-expression> ) ]
<simple-expression> ::= [ plus | minus ]
                        term
                        { ( <additive-operator> <term> ) }

<term> ::= <factor>

```



```

        { ( <multiplicative-operator> <factor> ) }

<factor> ::= ident | intcon | realcon | charcon | string
          | ( lparent <expression> rparent )
          | ( notsy <factor> )
          | <procedure/function-call>
          | <variable>

<relational-operator> ::= eql
                       | neq
                       | gtr
                       | geq
                       | lss
                       | leq

<additive-operator> ::= plus
                     | minus
                     | orsy

<multiplicative-operator> ::= times
                          | rdiv
                          | idiv
                          | imod
                          | andsy

```

2.3. Struktur Program

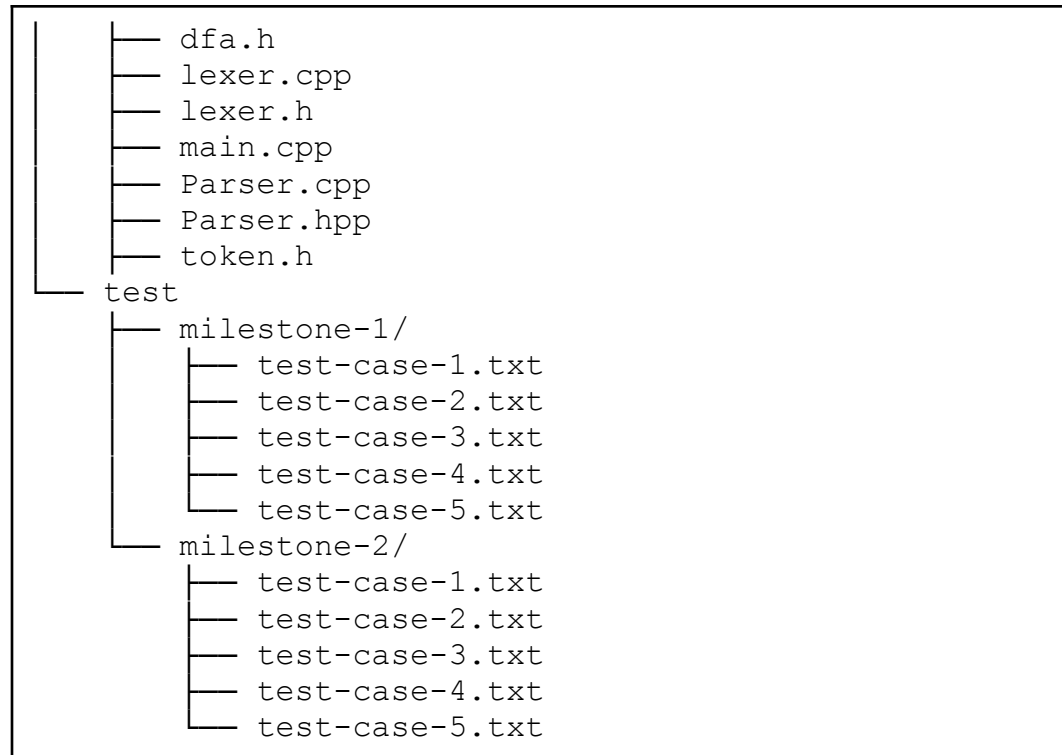
2.3.1. Struktur Folder

Dengan melanjutkan struktur pada Milestone 1, struktur folder pada Milestone 2 adalah sebagai berikut.

```

GTW-Tubes-IF2224
├── bin
│   └── lexer
├── doc/
│   └── Laporan-1-GTW.pdf
└── src/

```



2.3.2. Penjelasan File

Implementasi tambahan pada milestone ini adalah implementasi Parser disimpan dalam folder src melalui beberapa dua file berikut.

File	Peran
Parser.h	Mendeklarasikan kelas dan fungsi untuk parsing
Parser.cpp	Mengimplementasikan seluruh method untuk parsing

2.4. Struktur Data

2.4.1. Token

Token adalah satuan input yang diterima parser dari lexer. Setiap token memiliki dua atribut sebagai berikut.

```
struct Token
{
    string type;
    string value;
};
```

2.4.2. Parse Node

ParseNode adalah simpul dalam parse tree.

```
struct ParseNode{
    string type;
    string value;
    vector<shared_ptr<ParseNode>> children;

    ParseNode(const string& t): type(t) {};
    ParseNode(const string& t, const string& v): type(t),
value(v) {};
};
```

Terdapat dua jenis simpul yang diimplementasikan, antara lain:

- Node, yaitu simpul yang merepresentasikan non-terminal dan memiliki anak-anak
- Leaf, yaitu simpul yang merepresentasikan terminal dan tidak memiliki anak

```
// membuat node nonterminal
auto node = makeNode("<program>");

// membuat leaf terminal
auto leaf = makeLeaf(tok.type, tok.value);
```

2.5. Implementasi Parser

Parser diimplementasikan menggunakan pendekatan Recursive Descent Parsing, yaitu metode parsing top-down yang memetakan aturan grammar ke dalam fungsi-fungsi parser. Dengan pendekatan ini, setiap nonterminal dalam grammar direpresentasikan oleh satu fungsi. Misalnya, nonterminal `<program>` diimplementasikan oleh fungsi `parseProgram()`, `<program-header>` oleh `parseProgramHeader()`, dan `<subprogram-declaration>` oleh `parseSubProgramDeclaration()`.

Setiap fungsi parser bertanggung jawab untuk mengenali pola token sesuai dengan aturan produksi grammar yang bersangkutan. Apabila suatu aturan grammar

memuat nonterminal lain, fungsi parser akan memanggil fungsi parser lain yang sesuai. Dengan demikian, alur pemanggilan fungsi dalam program mengikuti struktur hierarkis grammar bahasa Arion.

Untuk mencocokkan token terminal, parser menggunakan fungsi `expect()`. Fungsi ini memeriksa apakah token saat ini sesuai dengan token yang diharapkan. Jika sesuai, token akan dikonsumsi dan dimasukkan sebagai leaf pada parse tree. Jika tidak sesuai, parser akan memanggil `syntaxError()` untuk menampilkan pesan kesalahan yang informatif.

Penjelasan implementasi parser pada bagian ini disusun berdasarkan pengelompokan nonterminal pada grammar bahasa Arion. Setiap kelompok nonterminal merepresentasikan bagian tertentu dalam struktur program, seperti program utama, deklarasi, tipe data, variabel, subprogram, block, statement, dan expression.

2.5.1. Program dan Program Header

Program merupakan titik awal proses parsing secara keseluruhan. Berdasarkan grammar, `<program>` diturunkan menjadi `<program-header>`, `<declaration-part>`, `<compound-statement>`, dan diakhiri dengan token period. Implementasi dilakukan melalui fungsi `parseProgram()`, yang memanggil setiap bagian secara berurutan. Setelah period berhasil dikonsumsi, parser memeriksa apakah seluruh token sudah habis menggunakan `isAtEnd()`. Jika masih terdapat token yang tersisa, parser melempar error karena input dianggap belum selesai secara sah.

```
shared_ptr<ParseNode> Parser::parseProgram() {
    auto node = makeNode("<program>");
    node->children.push_back(parseProgramHeader());
    node->children.push_back(parseDeclarationPart());
    node->children.push_back(parseCompoundStatement());
    node->children.push_back(expect("period"));

    if (!isAtEnd()) {
        syntaxError("end of program");
    }

    return node;
}
```

```
}
```

Fungsi `parseProgramHeader()` memproses header program dengan urutan token `programsy`, `ident` sebagai nama program, dan `semicolon` sebagai penutup header. Ketiga token tersebut bersifat wajib, sehingga ketiganya ditangani menggunakan `expect()`.

```
shared_ptr<ParseNode> Parser::parseProgramHeader() {
    auto node = makeNode("<program-header>");
    node->children.push_back(expect("programsy"));
    node->children.push_back(expect("ident"));
    node->children.push_back(expect("semicolon"));
    return node;
}
```

2.5.2. Declaration

Fungsi `parseDeclarationPart()` menangani seluruh bagian deklarasi yang ada sebelum `compound statement`. Bagian ini terdiri atas empat kelompok deklarasi yang diproses secara berurutan, yaitu deklarasi konstanta, deklarasi tipe, deklarasi variabel, dan deklarasi subprogram. Masing-masing kelompok menggunakan `loop while` sehingga dapat menangani lebih dari satu blok deklarasi dalam satu bagian yang sama. Deklarasi subprogram diperiksa terhadap dua kemungkinan token, yaitu `proceduresy` atau `functionsy`, sebelum memanggil `parseSubProgramDeclaration()`.

```
shared_ptr<ParseNode> Parser::parseDeclarationPart() {
    auto node = makeNode("<declaration-part>");
    while(check("constsy")) {
        node->children.push_back(parseConstDeclaration());
    }

    while(check("typesy")) {
        node->children.push_back(parseTypeDeclaration());
    }

    while(check("varsy")) {
```

```

        node->children.push_back(parseVarDeclaration());
    }

    while(check("proceduresy") || check("functionsy")) {
node->children.push_back(parseSubProgramDeclaration());
    }

    return node;
}

```

Fungsi `parseConstDeclaration()` memproses blok deklarasi konstanta yang diawali dengan token `constsy`. Setelah `constsy` dikonsumsi, parser memeriksa apakah token berikutnya adalah `ident`. Jika tidak ditemukan, parser melempar error karena minimal harus ada satu deklarasi konstanta. Setiap deklarasi konstanta terdiri atas `ident`, `eql`, nilai konstanta dari `parseConstant()`, dan semicolon penutup. Proses ini diulang selama token berikutnya masih berupa `ident`. Berbeda dengan `parseSubProgramDeclaration()`, fungsi ini menangani error recovery secara lokal menggunakan blok `try-catch`. Apabila terjadi error pada salah satu deklarasi konstanta, error dicatat ke dalam `errors` dan parser memanggil `synchronize()` agar dapat melanjutkan ke deklarasi berikutnya tanpa menghentikan proses parsing.

```

shared_ptr<ParseNode> Parser::parseConstDeclaration()
{
    auto node = makeNode("<const-declaration>");
    node->children.push_back(expect("constsy"));

    // Harus ada minimal satu ident
    if (!check("ident")) throw SyntaxError("Expected at least one
constant declaration after 'const'");
    while (check("ident"))
    {
        try
        {
            node->children.push_back(expect("ident"));
            node->children.push_back(expect("eql"));

```

```

        node->children.push_back(parseConstant());
        node->children.push_back(expect("semicolon"));
    }
    catch (const SyntaxError &e)
    {
        errors.push_back(e.what());
        synchronize();
    }
}
return node;
}

```

Parse Variable Declaration:

```
shared_ptr<ParseNode> Parser::parseVarDeclaration()
```

```

    auto node = makeNode("<var-declaration>");{

    node->children.push_back(expect("varsy"));

    if (!check("ident"))
    {
        syntaxError("ident (expected at least one variable
declaration after 'var')");
    }

    while (check("ident"))
    {
        node->children.push_back(parseIdentifierList());
        node->children.push_back(expect("colon"));
        node->children.push_back(parseType());
        node->children.push_back(expect("semicolon"));
    }

    return node;
}

```

2.5.3. Type

Parse Type Declaration:

```
shared_ptr<ParseNode> Parser::parseTypeDeclaration()
{
    auto node = makeNode("<type-declaration>");

    node->children.push_back(expect("typesy"));
    if (!check("ident"))
    {
        syntaxError("ident (expected at least one type
declaration after 'type')");
    }

    while (check("ident")) // lanjut selama masih ada ident
    berikutnya
    {
        node->children.push_back(expect("ident"));
        node->children.push_back(expect("eql"));
        node->children.push_back(parseType());
        node->children.push_back(expect("semicolon"));
    }

    return node;
}
```

Parse Type:

```
shared_ptr<ParseNode> Parser::parseType()
{
    auto node = makeNode("<type>");

    if (check("arraysy")) {
        node->children.push_back(parseArrayType());
    }
    else if (check("lparent")) {
```



```

        node->children.push_back(parseEnumerated());
    }
    else if (check("recordsy")) {
        node->children.push_back(parseRecordType());
    }
    else if (check("ident") && lookahead().type == "period") {
        auto firstConst = makeNode("<constant>");
        firstConst->children.push_back(consume()); // consume
ident
        node->children.push_back(parseRange(firstConst));
    }
    else if (check("intcon") || check("realcon") ||
check("charcon") ||
                                check("string") || check("plus") ||
check("minus"))
    {
        auto firstConst = parseConstant();
        if (check("period")) {
            node->children.push_back(parseRange(firstConst));
        }
        else {
            // Standalone constant (jarang, tapi valid secara
grammar)
            node->children.push_back(firstConst);
        }
    }
    else if (check("ident")) {
        node->children.push_back(expect("ident"));
    }
    else
    {
        syntaxError("type (ident, array-type, range,
enumerated, or record-type)");
    }

    return node;

```

```
}
```

Parse Array Type:

```
shared_ptr<ParseNode> Parser::parseArrayType()
{
    auto node = makeNode("<array-type>");

    node->children.push_back(expect("arraysy"));
    node->children.push_back(expect("lbrack"));

    if (check("ident") && lookahead().type == "period")
    {
        // Range diawali ident
        auto firstConst = makeNode("<constant>");
        firstConst->children.push_back(consume()); // consume
ident
        node->children.push_back(parseRange(firstConst));
    }
    else if (check("intcon") || check("charcon") ||
check("realcon") ||
        check("plus") || check("minus"))
    {
        // Range diawali intcon/charcon/etc
        auto firstConst = parseConstant();
        node->children.push_back(parseRange(firstConst));
    }
    else if (check("ident"))
    {
        node->children.push_back(expect("ident"));
    }
    else
    {
        syntaxError("range or ident as array index type");
    }

    node->children.push_back(expect("rbrack"));
```

```

node->children.push_back(expect("ofsy"));

// Element type, bisa nested array juga
node->children.push_back(parseType());

return node;
}

```

Parse Record Type:

```

shared_ptr<ParseNode> Parser::parseRecordType()
{
    auto node = makeNode("<record-type>");

    node->children.push_back(expect("recordsy"));
    node->children.push_back(parseFieldList());
    node->children.push_back(expect("endsy"));

    return node;
}

```

Parse Field List:

```

shared_ptr<ParseNode> Parser::parseFieldList()
{
    auto node = makeNode("<field-list>");

    node->children.push_back(parseFieldPart());

    while (check("semicolon") && lookahead().type == "ident")
    {
        node->children.push_back(expect("semicolon"));
        node->children.push_back(parseFieldPart());
    }

    return node;
}

```

Parse Field Part:

```
shared_ptr<ParseNode> Parser::parseFieldPart()
{
    auto node = makeNode("<field-part>");

    node->children.push_back(parseIdentifierList());
    node->children.push_back(expect("colon"));
    node->children.push_back(parseType());

    return node;
}
```

2.5.4. Range

Parse Range

```
shared_ptr<ParseNode> Parser::parseRange(shared_ptr<ParseNode>
firstConst)
{
    auto node = makeNode("<range>");
    node->children.push_back(firstConst);

    node->children.push_back(expect("period"));
    node->children.push_back(expect("period"));

    node->children.push_back(parseConstant());

    return node;
}
```

2.5.5. Enumerated

```
shared_ptr<ParseNode> Parser::parseEnumerated()
{
    auto node = makeNode("<enumerated>");
```

```

node->children.push_back(expect("lparent"));
node->children.push_back(expect("ident"));

while (check("comma"))
{
    node->children.push_back(expect("comma"));
    node->children.push_back(expect("ident"));
}

node->children.push_back(expect("rparent"));

return node;
}

```

2.5.6. Variable

Parse Variable:

```

shared_ptr<ParseNode> Parser::parseVariable()
{
    auto node = makeNode("<variable>");
    node->children.push_back(expect("ident"));

    while (check("lbrack") || check("period"))
    {
        node->children.push_back(parseComponentVariable());
    }

    return node;
}

```

Parse Component Variable:

```

shared_ptr<ParseNode> Parser::parseComponentVariable()
{
    auto node = makeNode("<component-variable>");

    if (check("lbrack"))
    {

```

```

        node->children.push_back(expect("lbrack"));
        node->children.push_back(parseIndexList());
        node->children.push_back(expect("rbrack"));
    }
    else if (check("period"))
    {
        node->children.push_back(expect("period"));
        node->children.push_back(expect("ident"));
    }
    else
    {
        syntaxError("'[' or '.' in component-variable");
    }

    return node;
}

```

Parse Index List:

```
shared_ptr<ParseNode> Parser::parseIndexList()
```

```

{
    auto node = makeNode("<index-list>");

    if (check("intcon") || check("charcon") || check("ident"))
        node->children.push_back(consume());
    else
        syntaxError("intcon, charcon, or ident as index");

    while (check("comma"))
    {
        node->children.push_back(expect("comma"));
        if (check("intcon") || check("charcon") ||
check("ident"))
            node->children.push_back(consume());
        else
            syntaxError("intcon, charcon, or ident as index");
    }
}

```

```
    return node;
}
```

Parse Identifier List:

```
shared_ptr<ParseNode> Parser::parseIdentifierList()
{
    auto node = makeNode("<identifier-list>");
    node->children.push_back(expect("ident"));

    while (check("comma") && lookahead().type == "ident") {
        node->children.push_back(expect("comma"));
        node->children.push_back(expect("ident"));
    }

    return node;
}
```

2.5.7. Subprogram

Subprogram digunakan untuk menangani deklarasi procedure dan function. Berdasarkan grammar, <subprogram-declaration> dapat diturunkan menjadi <procedure-declaration> atau <function-declaration>. Implementasi dilakukan melalui fungsi parseSubProgramDeclaration(), yaitu dengan memeriksa token saat ini. Jika token berupa proceduressy, parser memanggil parseProcedureDeclaration(), sedangkan jika token berupa functionsy, parser memanggil parseFunctionDeclaration().

Fungsi parseProcedureDeclaration() memproses deklarasi procedure dengan urutan proceduressy, ident, optional formal-parameter-list, semicolon, block, dan semicolon penutup. Formal parameter bersifat opsional, sehingga parseFormalParameterList() hanya dipanggil apabila token berikutnya adalah lparent.

Fungsi parseFunctionDeclaration() memiliki struktur yang mirip dengan procedure, tetapi memiliki tambahan colon dan ident sebagai tipe nilai balik.

Setelah header function selesai diproses, parser memanggil `parseBlock()` untuk membaca isi function.

Pada implementasi ini, `parseProcedureDeclaration()` dan `parseFunctionDeclaration()` tidak menangani error recovery secara lokal. Apabila terjadi error pada parameter, semicolon, colon, return type, atau block, error akan dilempar ke level pemanggil. Recovery subprogram dilakukan pada `parseDeclarationPart()` agar parser dapat melanjutkan proses parsing ke deklarasi subprogram berikutnya.

```
/**
 * procedure-declaration | function-declaration
 */
shared_ptr<ParseNode> Parser::parseSubProgramDeclaration()
{
    auto node = makeNode("<subprogram-declaration>");

    if (check("proceduresy"))
    {
        node->children.push_back(parseProcedureDeclaration());
    }
    else if (check("functionsy"))
    {
        node->children.push_back(parseFunctionDeclaration());
    }
    else
    {
        syntaxError("proceduresy or functionsy after subprogram
declaration.");
    }

    return node;
}

/**
 * proceduresy + ident + (formal-parameter-list)? + semicolon +
block + semicolon
 */
```



```

shared_ptr<ParseNode> Parser::parseProcedureDeclaration()
{
    auto node = makeNode("<procedure-declaration>");

    node->children.push_back(expect("proceduresy"));
    node->children.push_back(expect("ident"));

    if (check("lparent"))
    {
        node->children.push_back(parseFormalParameterList());
    }

    node->children.push_back(expect("semicolon"));
    node->children.push_back(parseBlock());
    node->children.push_back(expect("semicolon"));

    return node;
}

/**
 * functionsy + ident + (formal-parameter-list)? + colon + ident +
 * semicolon + block + semicolon
 */
shared_ptr<ParseNode> Parser::parseFunctionDeclaration()
{
    auto node = makeNode("<function-declaration>");

    node->children.push_back(expect("functionsy"));
    node->children.push_back(expect("ident"));

    // formal-parameter-list optional
    if (check("lparent"))
    {
        node->children.push_back(parseFormalParameterList());
    }

    node->children.push_back(expect("colon"));
    node->children.push_back(expect("ident"));
}

```

```
node->children.push_back(expect("semicolon"));

node->children.push_back(parseBlock());
node->children.push_back(expect("semicolon"));

return node;
}
```

2.5.8. Block

Bagian block digunakan untuk merepresentasikan isi dari program utama, procedure, maupun function. Berdasarkan grammar bahasa Arion, <block> terdiri dari dua bagian utama, yaitu <declaration-part> dan <compound-statement>. Jadi, sebuah block dapat memiliki bagian deklarasi terlebih dahulu, kemudian dilanjutkan dengan statement utama yang berada di dalam begin dan end.

Implementasi block dilakukan melalui fungsi parseBlock(). Fungsi ini membuat node <block>, kemudian memanggil parseDeclarationPart() untuk membaca deklarasi lokal yang mungkin terdapat di dalam block. Deklarasi tersebut dapat berupa deklarasi konstanta, tipe, variabel, maupun subprogram lain. Setelah bagian deklarasi selesai diproses, parser memanggil parseCompoundStatement() untuk membaca statement yang berada di dalam begin dan end.

Fungsi parseBlock() tidak melakukan error recovery secara lokal. Apabila terjadi kesalahan pada declaration-part atau compound-statement, error akan dilempar ke fungsi pemanggil. Pendekatan ini digunakan agar error pada block subprogram dapat ditangani oleh mekanisme recovery subprogram di level declaration-part. Dengan begitu, ketika terdapat error pada satu procedure atau function, parser tetap dapat melakukan sinkronisasi dan melanjutkan proses parsing ke subprogram berikutnya.

Selain block, bagian ini juga mencakup implementasi formal parameter pada procedure dan function. Fungsi parseFormalParameterList() digunakan

untuk membaca daftar parameter formal yang berada di dalam tanda kurung. Fungsi ini diawali dengan mencocokkan token lparent, kemudian membaca minimal satu parameter-group. Jika ditemukan token semicolon, parser akan membaca parameter-group berikutnya. Proses ini terus dilakukan hingga parser menemukan token rparent sebagai penutup daftar parameter.

Sementara itu, fungsi parseParameterGroup() digunakan untuk membaca satu kelompok parameter. Berdasarkan grammar, parameter-group terdiri dari identifier-list, colon, dan tipe parameter. Tipe parameter dapat berupa ident atau array-type. Oleh karena itu, fungsi ini memanggil parseIdentifierList(), kemudian mencocokkan token colon, lalu memeriksa apakah token berikutnya berupa ident atau arraysy. Jika token berupa ident, parser akan mencocokkannya sebagai tipe parameter. Jika token berupa arraysy, parser akan memanggil parseArrayType().

Dengan implementasi ini, parser dapat memproses block pada program utama maupun subprogram, serta dapat membaca formal parameter pada procedure dan function, baik yang terdiri dari satu parameter maupun beberapa parameter-group yang dipisahkan oleh semicolon.

```
/**
 * declaration-part + compound-statement
 */
shared_ptr<ParseNode> Parser::parseBlock()
{
    auto node = makeNode("<block>");

    node->children.push_back(parseDeclarationPart());
    node->children.push_back(parseCompoundStatement());

    return node;
}

/**
 * lparent + parameter-group + (semicolon + parameter-group)* +
 rparent
 */
shared_ptr<ParseNode> Parser::parseFormalParameterList()
```

```

{
    auto node = makeNode("<formal-parameter-list>");

    node->children.push_back(expect("lparent"));
    node->children.push_back(parseParameterGroup());

    while (check("semicolon"))
    {
        node->children.push_back(expect("semicolon"));
        node->children.push_back(parseParameterGroup());
    }

    node->children.push_back(expect("rparent"));

    return node;
}

/**
 * identifier-list + colon + (ident | array-type)
 */
shared_ptr<ParseNode> Parser::parseParameterGroup()
{
    auto node = makeNode("<parameter-group>");

    node->children.push_back(parseIdentifierList());
    node->children.push_back(expect("colon"));

    if (check("ident"))
    {
        node->children.push_back(expect("ident"));
    }
    else if (check("arraysy"))
    {
        node->children.push_back(parseArrayType());
    }
    else
    {
        syntaxError("ident or array-type");
    }
}

```

```

    }

    return node;
}

```

2.5.9. Statement

Parsing statement ini terdapat beberapa fungsi yang mendekomposisi sampai jenis terkecil dari statement. Fungsi `parseCompoundStatement()` berangkat dengan token `begin` di awal dan token `end` di akhir. Di tengah kedua token tersebut, fungsi `parseStatementList()` dipanggil untuk memproses setiap statement dengan jenisnya masing-masing. Fungsi `parseStatementList()` melanjutkan fungsi sebelumnya yang berisi kumpulan fungsi `parseStatement()` sampai bertemu token `end`, `until`, atau di akhir file.

Pada fungsi `parseStatement()`, fungsi tersebut akan memanggil setiap jenis statement yang bersesuaian dengan karakter dari setiap jenisnya. Misal `parseIfStatement` dengan `if` di awal, `parseAssignmentStatement()` dengan token `becomes` pada setelah `ident` dan lainnya. Setiap fungsi `parse` tersebut akan memproses setiap tokennya sesuai dengan spesifikasi yang diberikan. Setiap kesalahan atau ketidaksesuaian dengan spek akan terdapat `syntax error`.

Parse Compound Statement:

```

shared_ptr<ParseNode> Parser::parseCompoundStatement()
{
    auto node = makeNode("<compound-statement>");

    node->children.push_back(expect("beginsy"));
    node->children.push_back(parseStatementList());
    node->children.push_back(expect("endsy"));

    return node;
}

```

Parse Statement List:

```
shared_ptr<ParseNode> Parser::parseStatementList()
{
    auto node = makeNode("<statement-list>");
    if (check("endsy") || check("untilsy") || isAtEnd())
        return node;
    node->children.push_back(parseStatement());
    while (check("semicolon"))
    {
        node->children.push_back(expect("semicolon"));

        if (check("endsy") || check("untilsy") || isAtEnd())
            break;

        node->children.push_back(parseStatement());
    }
    return node;
}
```

Parse Statement:

```
shared_ptr<ParseNode> Parser::parseStatement()
{
    auto node = makeNode("<statement>");
    if (check("ifsy"))
    {
        node->children.push_back(parseIfStatement());
    }
    else if (check("casesy"))
    {
        node->children.push_back(parseCaseStatement());
    }
    else if (check("whilesy"))
```

```

{
    node->children.push_back(parseWhileStatement());
}
else if (check("repeatsy"))
{
    node->children.push_back(parseRepeatStatement());
}
else if (check("forsy"))
{
    node->children.push_back(parseForStatement());
}
else if (check("ident"))
{
    auto identLeaf = parseVariable();

    if (check("becomes"))

node->children.push_back(parseAssignmentStatement(identLeaf));
    else

node->children.push_back(parseProcedureFunctionCall(identLeaf)
);
}

return node;
}

```

Parse Assignment Statement:

```

shared_ptr<ParseNode>
Parser::parseAssignmentStatement(shared_ptr<ParseNode>
identLeaf)
{
    auto node = makeNode("<assignment-statement>");
    node->children.push_back(identLeaf);
}

```

```

node->children.push_back(expect("becomes"));
node->children.push_back(parseExpression());
return node;
}

```

Parse If Statement:

```

shared_ptr<ParseNode> Parser::parseIfStatement()
{
    auto node = makeNode("<if-statement>");
    node->children.push_back(expect("ifsy"));
    node->children.push_back(parseExpression());
    node->children.push_back(expect("thensy"));
    node->children.push_back(parseStatement());
    if (check("elsesy"))
    {
        node->children.push_back(expect("elsesy"));

        node->children.push_back(parseStatement());
    }
    return node;
}

```

Parse Case Statement:

```

shared_ptr<ParseNode> Parser::parseCaseStatement()
{
    auto node = makeNode("<case-statement>");
    node->children.push_back(expect("casesy"));
    node->children.push_back(parseExpression());
    node->children.push_back(expect("ofsy"));
    node->children.push_back(parseCaseBlock());

    while (check("semicolon") && !isAtEnd())
    {

```



```

        node->children.push_back(expect("semicolon"));
        if (check("endsy"))
            break;
        node->children.push_back(parseCaseBlock());
    }

    node->children.push_back(expect("endsy"));
    return node;
}

```

Parse Case Block:

```

shared_ptr<ParseNode> Parser::parseCaseBlock()
{
    auto node = makeNode("<case-block>");
    node->children.push_back(parseConstant());
    while (check("comma"))
    {
        node->children.push_back(expect("comma"));

        node->children.push_back(parseConstant());
    }
    node->children.push_back(expect("colon"));
    node->children.push_back(parseStatement());
    return node;
}

```

Parse While Statement:

```

shared_ptr<ParseNode> Parser::parseWhileStatement()
{
    auto node = makeNode("<while-statement>");
    node->children.push_back(expect("whilesy"));
    node->children.push_back(parseExpression());
}

```

```

node->children.push_back(expect("dosy"));
node->children.push_back(parseStatement());
return node;
}

```

Parse Repeat Statement:

```

shared_ptr<ParseNode> Parser::parseRepeatStatement()
{
    auto node = makeNode("<repeat-statement>");
    node->children.push_back(expect("repeatsy"));
    node->children.push_back(parseStatementList());
    node->children.push_back(expect("untilsy"));
    node->children.push_back(parseExpression());
    return node;
}

```

Parse For Statement:

```

shared_ptr<ParseNode> Parser::parseForStatement()
{
    auto node = makeNode("<for-statement>");
    node->children.push_back(expect("forsy"));
    node->children.push_back(expect("ident"));
    node->children.push_back(expect("becomes"));
    node->children.push_back(parseExpression());

    if (check("tosy"))
        node->children.push_back(expect("tosy"));
    else if (check("downtosy"))
        node->children.push_back(expect("downtosy"));
    else
        syntaxError("tosy or downtosy");

    node->children.push_back(parseExpression());
}

```

```

node->children.push_back(expect("dosy"));
node->children.push_back(parseStatement());
return node;
}

```

Parse Procedure Function Call:

```

shared_ptr<ParseNode>
Parser::parseProcedureFunctionCall(shared_ptr<ParseNode>
identLeaf)
{
    auto node = makeNode("<procedure/function-call>");
    node->children.push_back(identLeaf);
    if (check("lparent"))
    {
        node->children.push_back(expect("lparent"));
        if (!check("rparent"))
        {
            node->children.push_back(parseParameterList());
        }
        node->children.push_back(expect("rparent"));
    }
    return node;
}

```

Parse Parameter List:

```

shared_ptr<ParseNode> Parser::parseParameterList()
{
    auto node = makeNode("<parameter-list>");
    node->children.push_back(parseExpression());
    while (check("comma"))
    {
        node->children.push_back(expect("comma"));
        node->children.push_back(parseExpression());
    }
}

```

```

    }

    return node;
}

```

2.5.10. Expression

Fungsi `parseConstant()` memproses nilai konstanta dalam tiga bentuk. Jika token berupa `charcon` atau `string`, token langsung dikonsumsi. Jika token berupa plus atau minus, parser mengonsumsi tanda tersebut lalu memeriksa token berikutnya yang harus berupa `intcon`, `realcon`, atau `ident`, dan melempar error jika tidak ditemukan. Jika token langsung berupa `intcon`, `realcon`, atau `ident` tanpa tanda, token dikonsumsi secara langsung. Apabila tidak ada kemungkinan yang cocok, parser melempar error. Fungsi ini tidak menangani error recovery secara lokal sehingga penanganannya diserahkan ke `parseConstDeclaration()`.

```

shared_ptr<ParseNode> Parser::parseConstant() {
    auto node = makeNode("<constant>");
    if(check("string") || check("charcon")){
        node->children.push_back(consume());
    }
    else if(check("plus") || check("minus")){
        node->children.push_back(consume());
        if(check("intcon") || check("realcon") ||
check("ident")){
            node->children.push_back(consume());
        }
        else {
            syntaxError("ident, intcon, or realcon adter sign
in constant");
        }
    }
    else if (check("intcon") || check("realcon") ||
check("ident")){
        node->children.push_back(consume());
    }
}

```

```

    }

    else{
        syntaxError("constant (charcon, string, intcon,
realcon, or signed value)");
    }

    return node;
}

```

Bagian selanjutnya menangani <expression>, <simple-expression>, <term>, dan <factor>.

```

shared_ptr<ParseNode> Parser::parseExpression()
{
    auto node = makeNode("<expression>");
    try {
        node->children.push_back(parseSimpleExpression());

        if (check("eq1") || check("neq") || check("gtr") ||
check("geq") || check("lss") || check("leq"))
        {
            node->children.push_back(parseRelationalOperator());
            node->children.push_back(parseSimpleExpression());
        }
    } catch (const SyntaxError &e)
    {
        errors.push_back(e.what());
        synchronize();
    } return node;
}

```

```

shared_ptr<ParseNode> Parser::parseSimpleExpression()
{
    auto node = makeNode("<simple-expression>");

    try {

```

```

        if (check("plus") || check("minus"))
        {
            node->children.push_back(consume());
        }

        node->children.push_back(parseTerm());

        while (check("plus") || check("minus") || check("orsy"))
        {
            node->children.push_back(parseAdditiveOperator());
            node->children.push_back(parseTerm());
        }
    } catch (const SyntaxError &e)
    {
        errors.push_back(e.what());
        synchronize();
    }
    return node;
}

```

```

shared_ptr<ParseNode> Parser::parseTerm()
{
    auto node = makeNode("<term>");

    node->children.push_back(parseFactor());

    while (check("times") || check("idiv") || check("rdiv") ||
check("imod") || check("andsy"))
    {
        node->children.push_back(parseMultiplicativeOperator());
        node->children.push_back(parseFactor());
    }

    return node;
}

```

```

shared_ptr<ParseNode> Parser::parseFactor()
{

```

```

auto node = makeNode("<factor>");

    if (check("intcon") || check("realcon") || check("charcon") ||
check("string"))
        node->children.push_back(consume());
    else if (check("ident"))
    {
        auto identLeaf = consume();
        if (check("lbrack") || check("period"))
        {
            auto varNode = makeNode("<variable>");
            varNode->children.push_back(identLeaf);
            while (check("lbrack") || check("period"))
varNode->children.push_back(parseComponentVariable());
            node->children.push_back(varNode);
        }
        else if (check("lparent"))
        {
node->children.push_back(parseProcedureFunctionCall(identLeaf));
        }
        else
        {
            auto varNode = makeNode("<variable>");
            varNode->children.push_back(identLeaf);
            node->children.push_back(varNode);
        }
    }
    else if (check("lparent"))
    {
        node->children.push_back(consume());
        node->children.push_back(parseExpression());
        node->children.push_back(expect("rparent"));
    }
    else if (check("notsy"))
    {
        node->children.push_back(consume());
        node->children.push_back(parseFactor());
    }

```

```

    }
    else // cek apakah dia
variable? or function-call
        node->children.push_back(parseVariable()); // Asumsi
pengecekan variable (other opt using ident)

    return node;
}

```

Bagian ini menangani <relational-operator>, <additive-operator>, dan <multiplicative-operator>. <relational-operator> menerima token eql | neq | gtr | geq | lss | leq sebagai token valid, jika tidak mengembalikan syntax error. <additive-operator> menerima token plus | minus | orsy sebagai token valid, jika tidak mengembalikan syntax error. <multiplicative-operator> menerima times | rdiv | idiv | imod | andsy sebagai token valid, jika tidak mengembalikan syntax error.

```

shared_ptr<ParseNode> Parser::parseRelationalOperator()
{
    auto node = makeNode("<relational-operator>");

    if (check("eql") || check("neq") || check("gtr") || check("geq")
|| check("lss") || check("leq"))
        node->children.push_back(consume());
    else
        syntaxError("relational-operator");
    return node;
}

```

```

shared_ptr<ParseNode> Parser::parseAdditiveOperator()
{
    auto node = makeNode("<additive-operator>");

    if (check("plus") || check("minus") || check("orsy"))
        node->children.push_back(consume());
    else
        syntaxError("additive-operator");
}

```



```

        return node;
    }

shared_ptr<ParseNode> Parser::parseMultiplicativeOperator()
{
    auto node = makeNode("<multiplicative-operator>");

    if (check("times") || check("idiv") || check("rdiv") ||
check("imod") || check("andsy"))
        node->children.push_back(consume());
    else
        syntaxError("multiplicative-operator");
    return node;
}

```

2.6. Error Handling

Bagian ini ditujukan untuk menangani token yang tidak sesuai dengan grammar. Parser akan memanggil fungsi ini ketika struktur token tidak sesuai dengan grammar yang dimiliki oleh bahasa pemrograman. Pesan *error* kemudian akan disimpan pada suatu struktur untuk dikeluarkan setelah menemukan seluruh *error* pada program.

```

void Parser::syntaxError(const string &expected)
{
    const Token &tok = current();
    ostringstream oss;
    oss << "Syntax error at line " << tok.line << ", col " << tok.col <<
": unexpected token '" << tok.type;
    if (!tok.value.empty())
        oss << "(" << tok.value << ")";
    oss << ", expected " << expected;
    throw SyntaxError(oss.str());
}

```

BAB III

PENGUJIAN

3.1. Metode Pengujian

Serangkaian pengujian terhadap parser untuk memvalidasi kemampuan program dalam mengenali berbagai jenis node parsing pada bahasa Arion. Pengujian dilakukan dengan memberikan masukan beberapa berkas masukan .txt yang unik agar dapat memastikan pemenuhan fungsionalitas program secara keseluruhan. Masing-masing kasus uji memuat berbagai bentuk input dan kondisi error.

3.2. Hasil Pengujian

3.2.1. Kasus Uji 1

- **Kasus:** Program valid dengan berbagai jenis konstanta (intcon, realcon, charcon, string, empty string, konstanta bertanda +/-, dan alias identifier)
- **Tujuan:** Memverifikasi parser dapat memproses program Arion yang syntactically valid tanpa menghasilkan error apapun
- **Hasil:** **Lulus**

Kode Program::

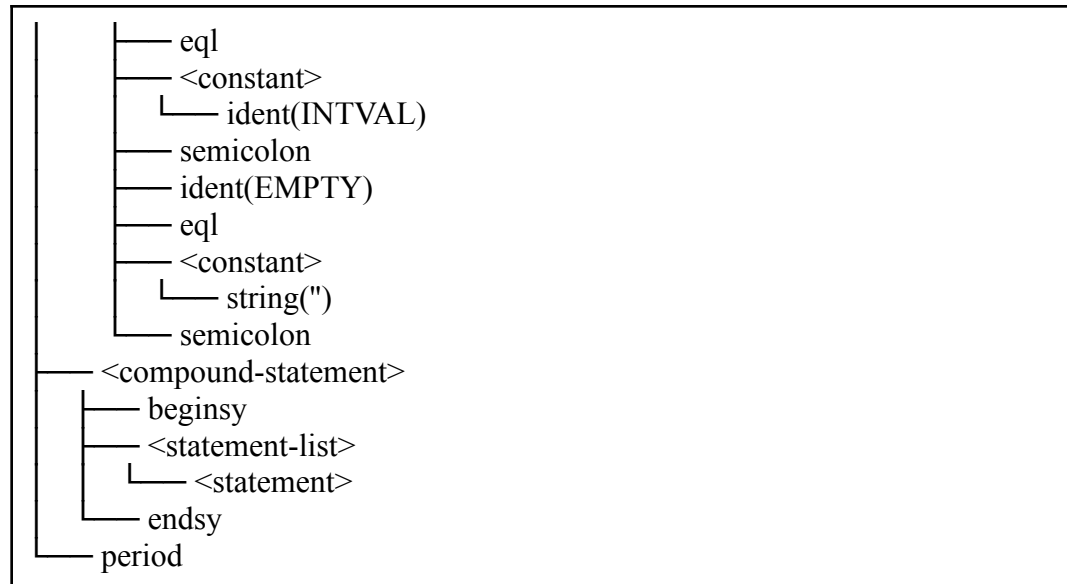
```
program ValidTest;

const
  INTVAL == 100;
  REALVAL == 3.14;
  CHARVAL == 'a';
  STRVAL == 'hello world';
  POSVAL == +99;
  NEGVAL == -42;
  ALIAS == INTVAL;
  EMPTY == "";
```

begin
end.

Hasil Uji:

```
<program>
├── <program-header>
│   ├── programsy
│   ├── ident(ValidTest)
│   └── semicolon
├── <declaration-part>
│   └── <const-declaration>
│       ├── constsy
│       ├── ident(INTVAL)
│       ├── eql
│       ├── <constant>
│       │   └── intcon(100)
│       ├── semicolon
│       ├── ident(REALVAL)
│       ├── eql
│       ├── <constant>
│       │   └── realcon(3.14)
│       ├── semicolon
│       ├── ident(CHARVAL)
│       ├── eql
│       ├── <constant>
│       │   └── charcon('a')
│       ├── semicolon
│       ├── ident(STRVAL)
│       ├── eql
│       ├── <constant>
│       │   └── string('hello world')
│       ├── semicolon
│       ├── ident(POSVAL)
│       ├── eql
│       ├── <constant>
│       │   ├── plus
│       │   └── intcon(99)
│       ├── semicolon
│       ├── ident(NEGVAL)
│       ├── eql
│       ├── <constant>
│       │   └── intcon(-42)
│       ├── semicolon
│       └── ident(ALIAS)
```



3.2.2. Kasus Uji 2

- **Kasus:** Program tanpa keyword program, langsung diawali identifier Test
- **Tujuan:** Memverifikasi parser mendeteksi error ketika program header tidak dimulai dengan keyword program
- **Hasil:** **Lulus**

Kode Program::

```

{ ERROR: tidak ada keyword program }
Test;
begin
end.

```

Hasil Uji:

Syntax error at line 2, col 5: unexpected token 'ident(Test)', expected programsy

3.2.3. Kasus Uji 3

- **Kasus:** Keyword program ada tetapi nama program tidak ada, langsung diikuti semicolon
- **Tujuan:** Memverifikasi parser mendeteksi error ketika identifier nama program tidak ditemukan setelah keyword program

- Hasil: **Lulus**

Kode Program::

```
{ ERROR: tidak ada nama program }  
program ;  
begin  
end.
```

Hasil Uji:

Syntax error at line 2, col 10: unexpected token 'semicolon(;)', expected ident

3.2.4. Kasus Uji 4

- **Kasus:** Program header valid tetapi tidak ada begin ... end sama sekali, file langsung habis
- **Tujuan:** Memverifikasi parser mendeteksi error ketika compound statement tidak ditemukan dan token sudah mencapai EOF
- Hasil: **Lulus**

Kode Program::

```
{ ERROR: tidak ada compound statement sama sekali }  
program Test;
```

Hasil Uji:

Syntax error at line 0, col 0: unexpected token 'eof', expected beginsy

3.2.5. Kasus Uji 5

- **Kasus:** Blok const dengan tiga jenis error berbeda: nilai kosong setelah ==, tidak ada operator ==, dan tanda tanpa nilai setelahnya
- **Tujuan:** Memverifikasi kemampuan error recovery parser dalam mengumpulkan multiple syntax errors sekaligus tanpa berhenti di error pertama
- Hasil: **Lulus**

Kode Program::

```
{ ERROR: const tanpa nilai setelah == }  
program Test;  
const  
    X ==;  
    X 100;  
    X == +;  
begin  
end.
```

Hasil Uji:

Syntax error at line 4, col 10: unexpected token 'semicolon(;)', expected constant (charcon, string, intcon, realcon, or signed value)
Syntax error at line 5, col 10: unexpected token 'intcon(100)', expected eql
Syntax error at line 6, col 12: unexpected token 'semicolon(;)', expected ident, intcon, or realcon after sign in constant

3.2.6. Kasus Uji 6

- **Kasus:** Program Arion lengkap yang mencakup seluruh konstruksi yang diimplementasikan: type declaration (range intcon, range charcon, enumerated, array dengan range intcon/charcon/ident, record sederhana/dengan field array/nested/kompleks), var declaration (identifier tunggal, identifier-list, berbagai tipe), serta compound statement dengan assignment ke variable biasa, array dengan index intcon/ident, field record, dan akses berantai field record lalu array.
- **Tujuan:** Memverifikasi bahwa seluruh fungsi parseTypeDeclaration, parseVarDeclaration, parseIdentifierList, parseType, parseArrayType, parseRange, parseEnumerated, parseRecordType, parseFieldList, parseFieldPart, parseVariable, parseComponentVariable, dan parseIndexList bekerja dengan benar secara terintegrasi.
- **Hasil:** **Lulus**

Kode Program:

```
program TestComprehensive;

type
    SmallInt == 1..100;
    LowerCase == 'a'..'z';
    Direction == (North, South, East, West);
    IntArray == array[1..10] of integer;
    CharArray == array['a'..'z'] of integer;
    DirArray == array[SmallInt] of integer;
    Point == record
        x, y: integer
    end;
    Matrix == record
        data: array[1..10] of integer;
        size: integer
    end;
    Line == record
        start, finish: Point;
        length: real
    end;
    Student == record
        name: string;
        age: integer;
        gpa: real;
        initial: char;
        grades: array[1..5] of integer
    end;

var
    a: integer;
    b, c, d: integer;
    p: Point;
```

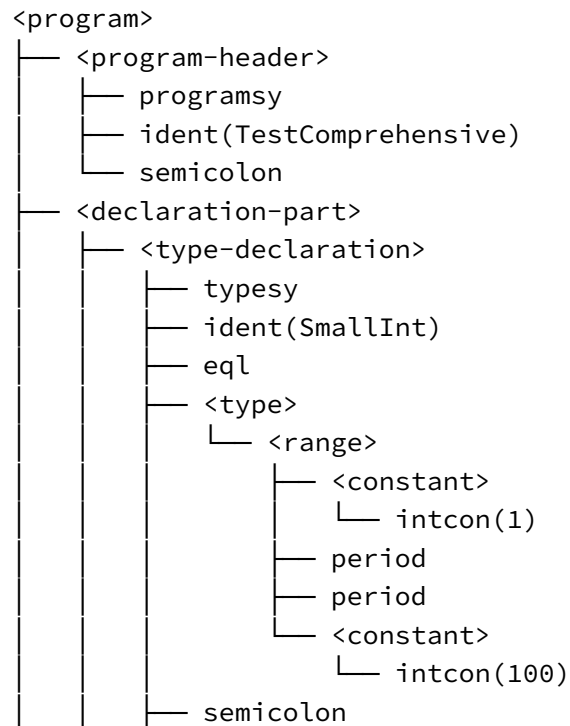
```

s: Student;
nums: array[1..10] of integer;
arr: array[SmallInt] of real;
dir: Direction;

begin
  a := 5;
  nums[1] := 10;
  nums[a] := 20;
  arr[a] := 3;
  p.x := 100;
  p.y := 200;
  s.age := 21;
  s.grades[1] := 90;
  nums[1] := nums[a]
end.

```

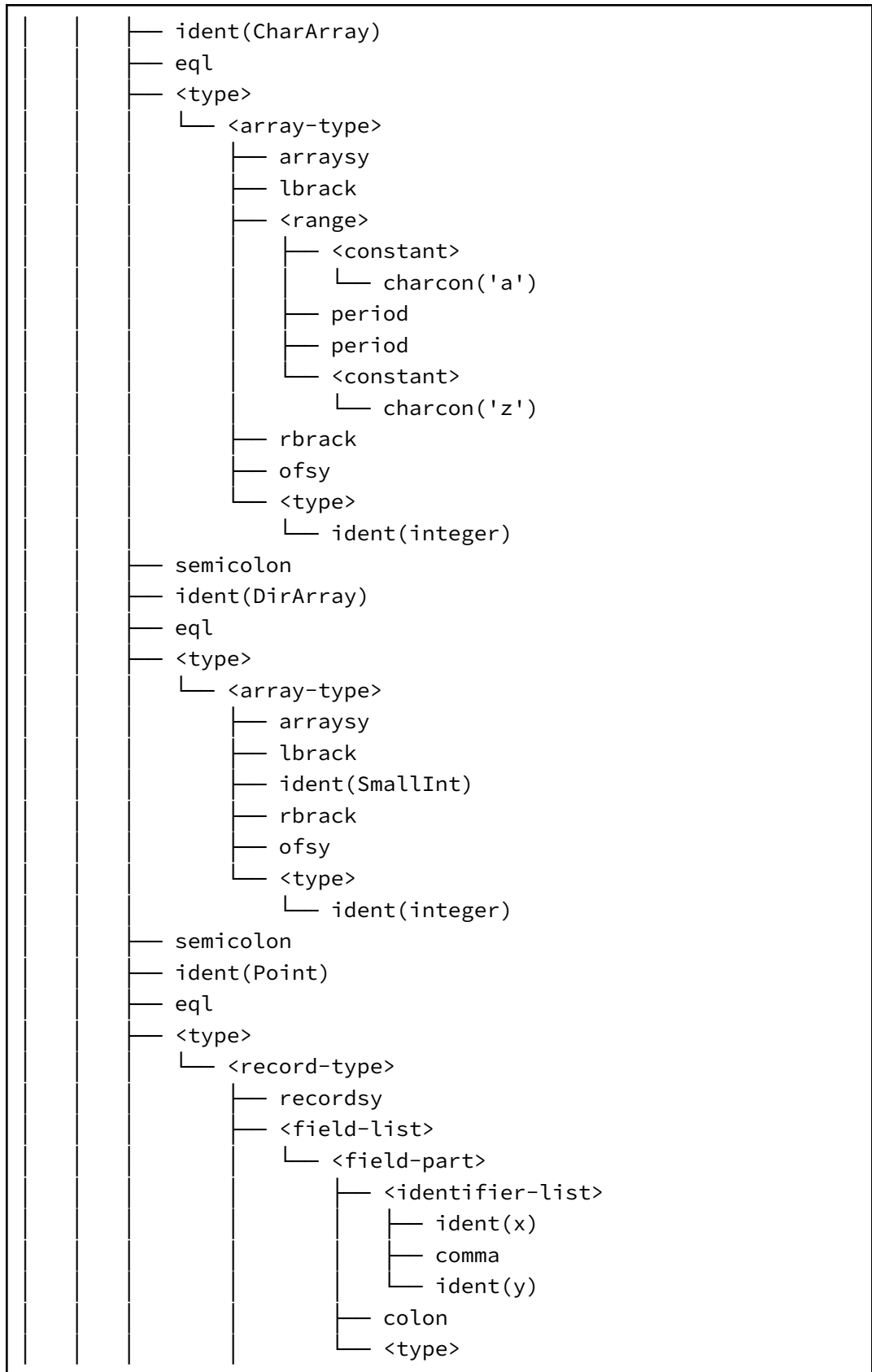
Hasil Uji:

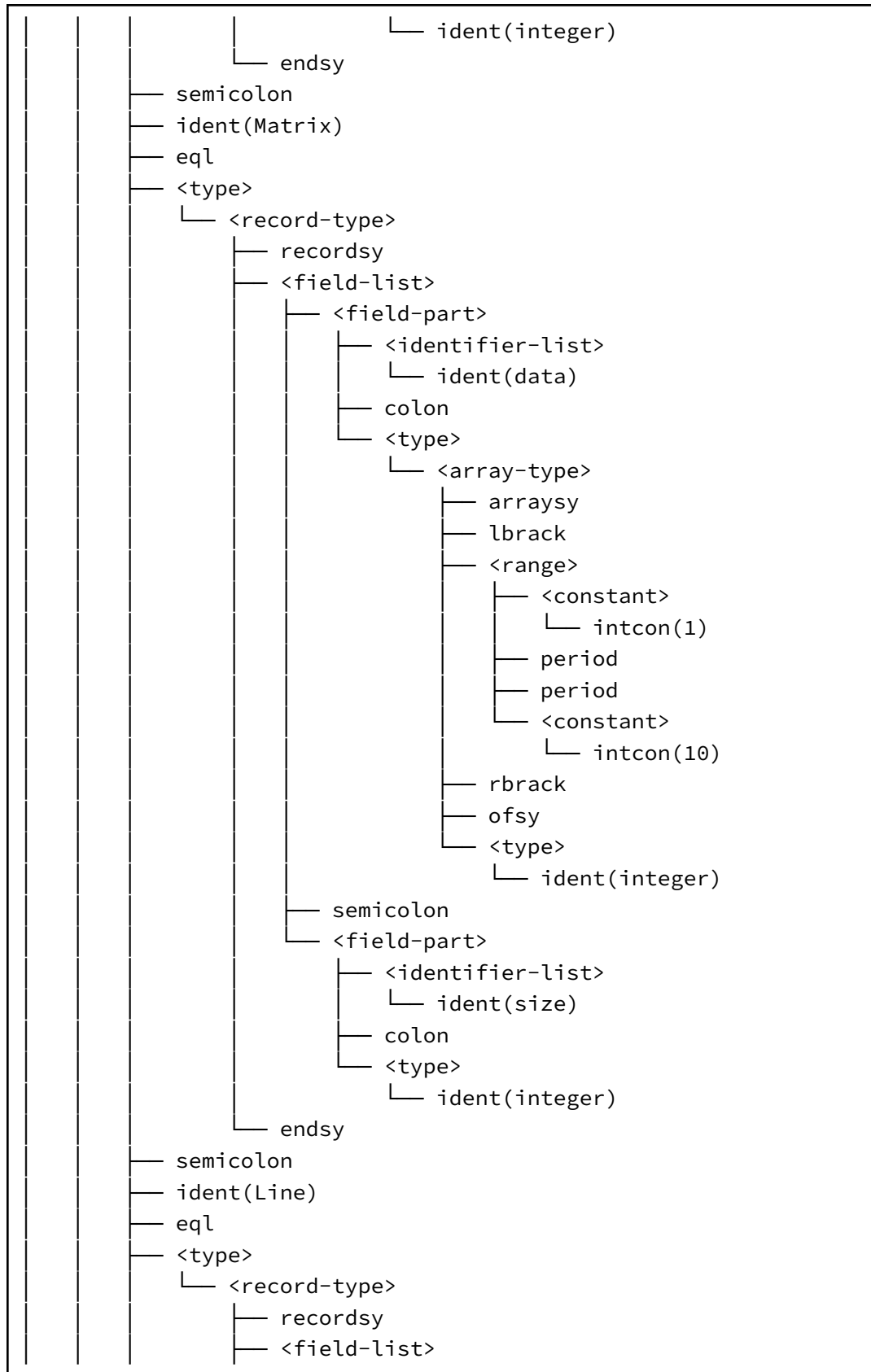


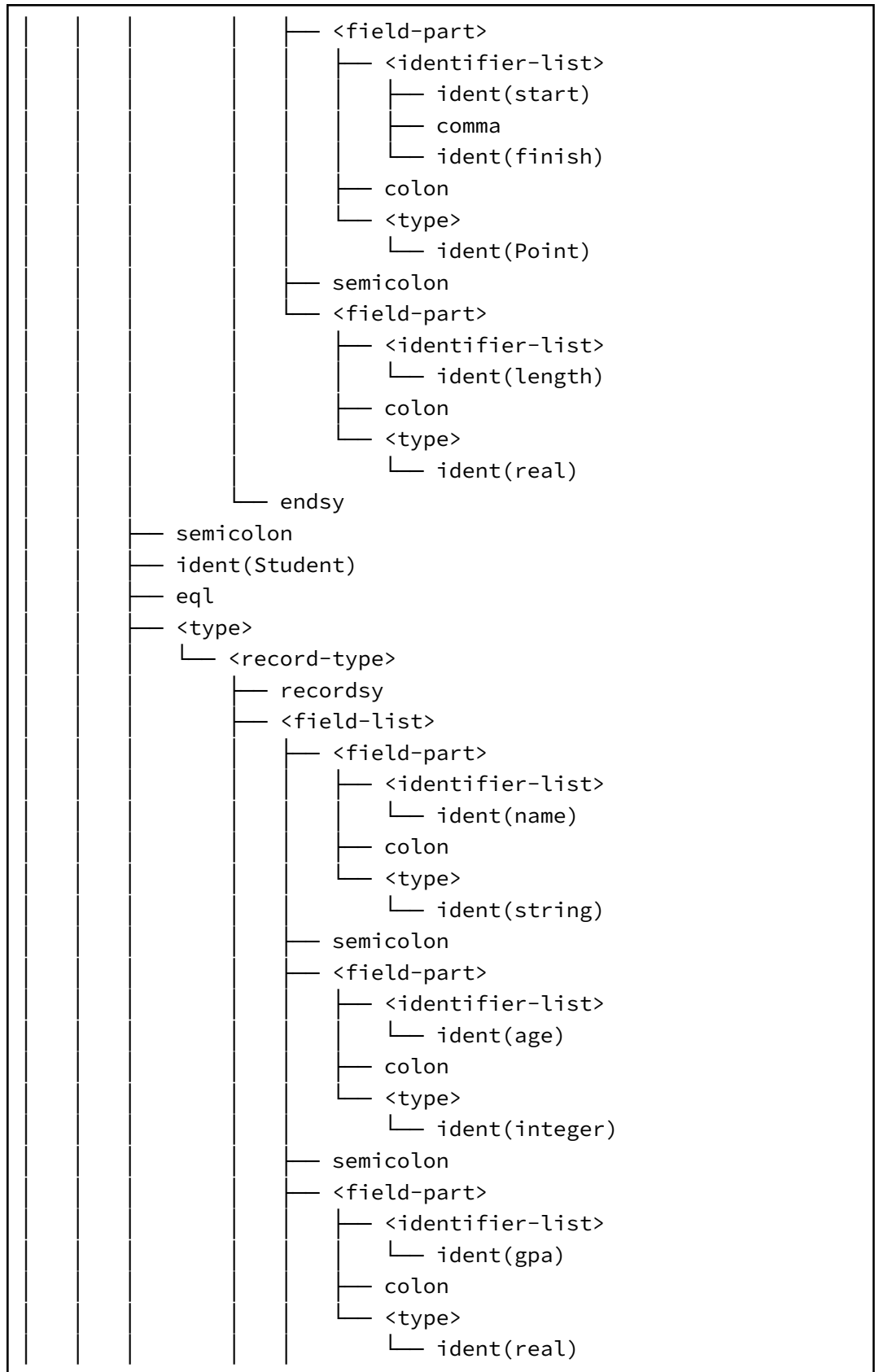

```

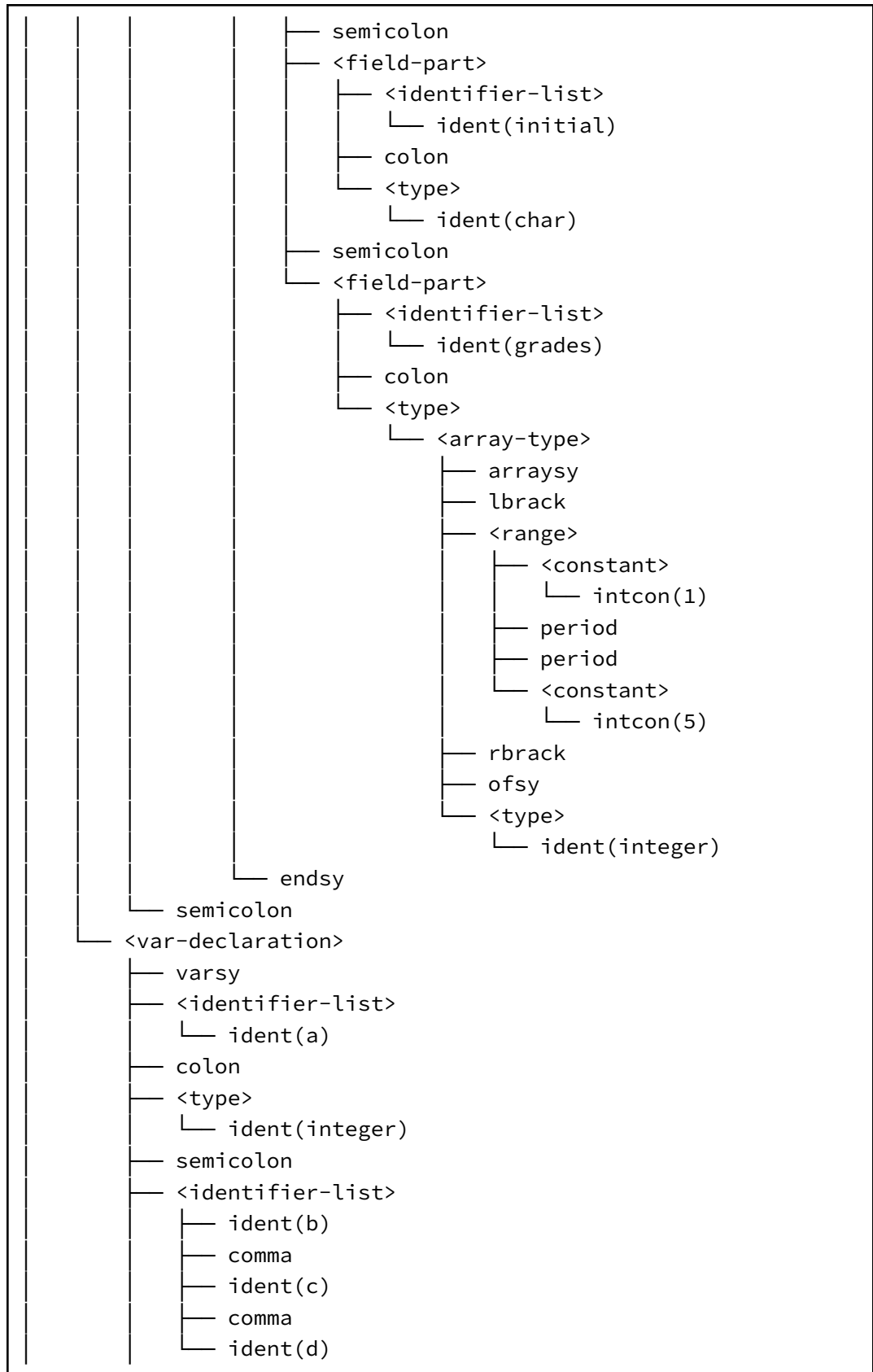
— ident(LowerCase)
— eql
— <type>
  — <range>
    — <constant>
      — charcon('a')
    — period
    — period
    — <constant>
      — charcon('z')
— semicolon
— ident(Direction)
— eql
— <type>
  — <enumerated>
    — lparent
    — ident(North)
    — comma
    — ident(South)
    — comma
    — ident(East)
    — comma
    — ident(West)
    — rparent
— semicolon
— ident(IntArray)
— eql
— <type>
  — <array-type>
    — arraysy
    — lbrack
    — <range>
      — <constant>
        — intcon(1)
      — period
      — period
      — <constant>
        — intcon(10)
    — rbrack
    — ofsy
    — <type>
      — ident(integer)
— semicolon

```





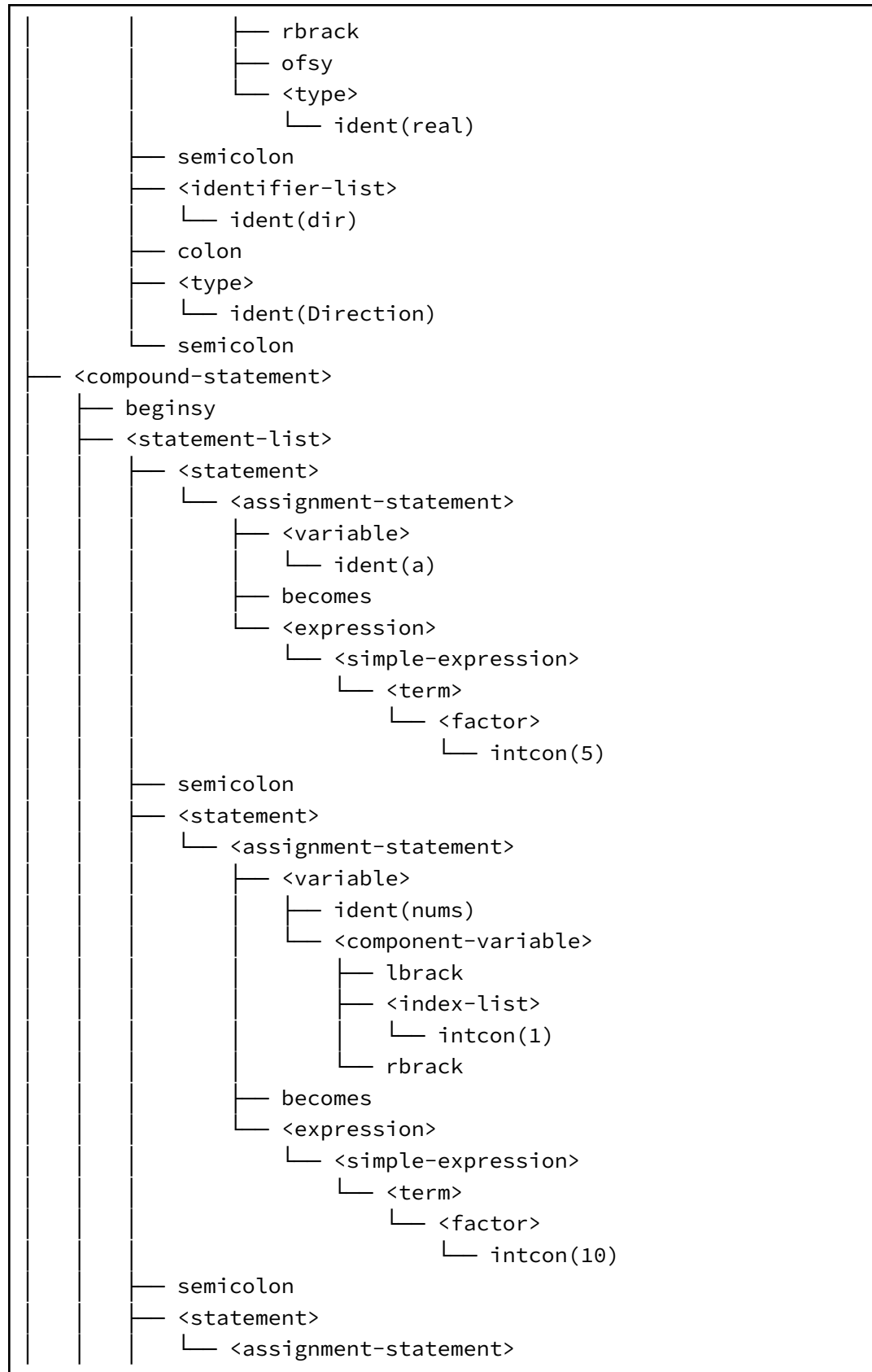


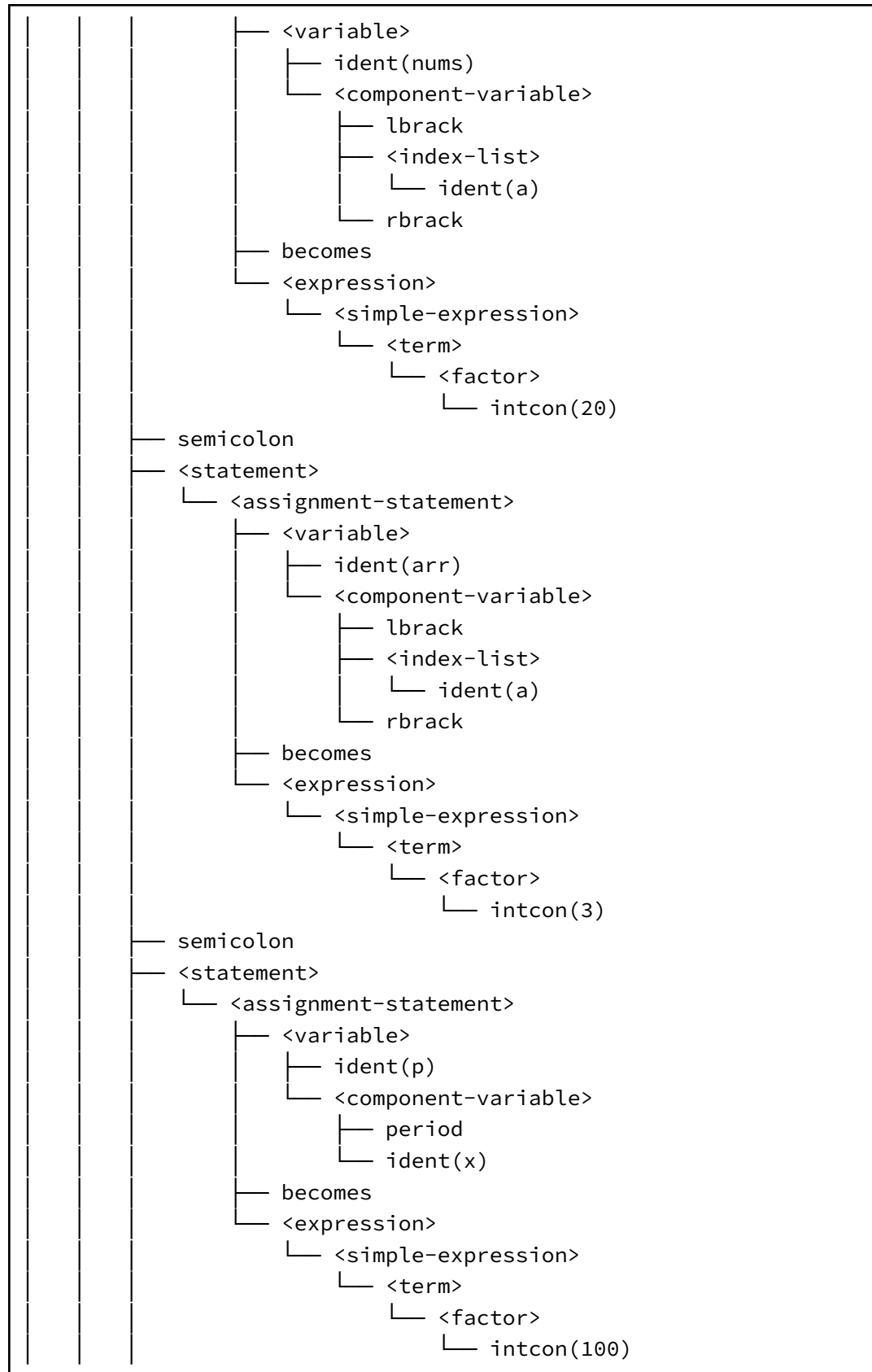


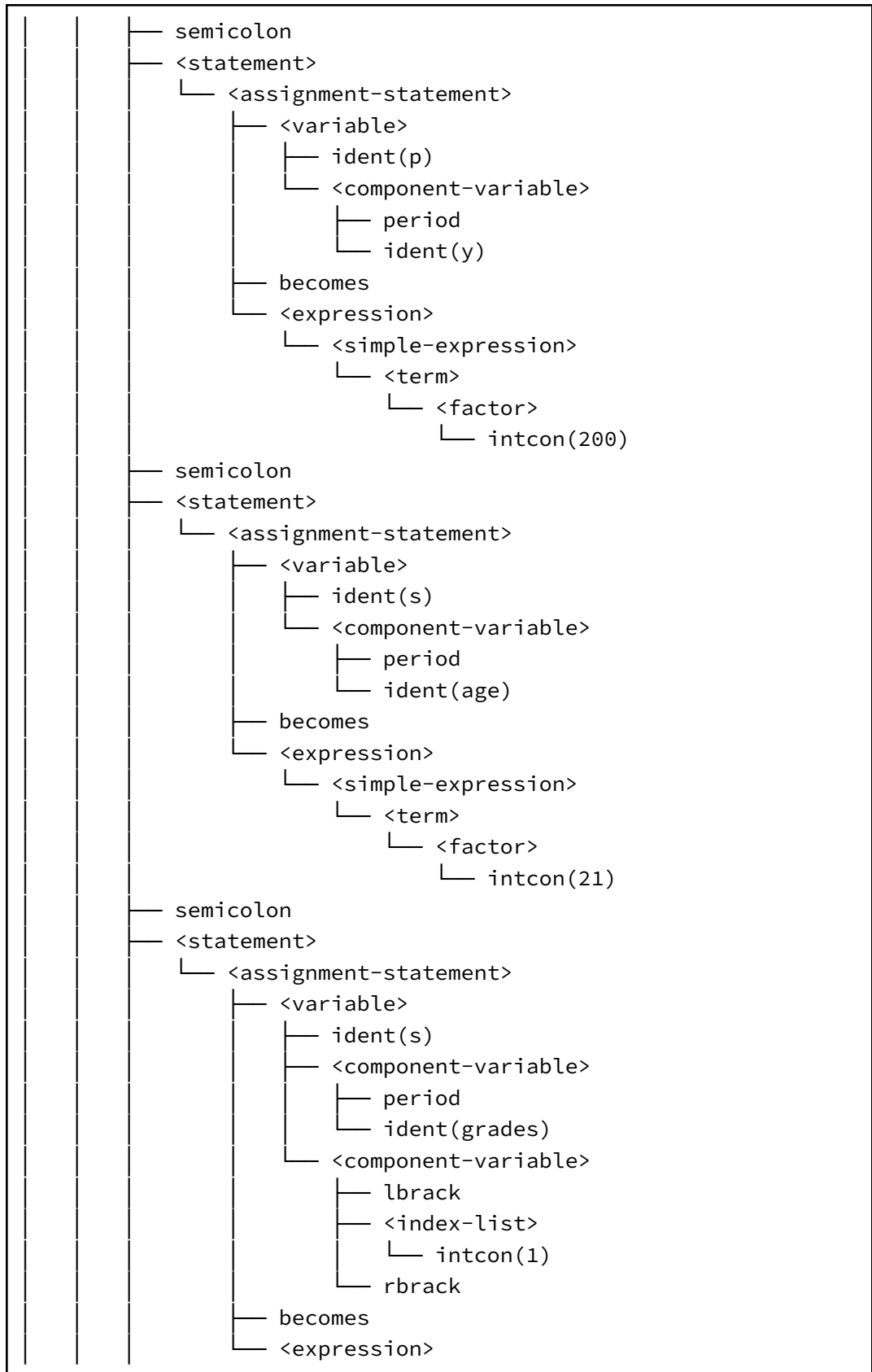
```

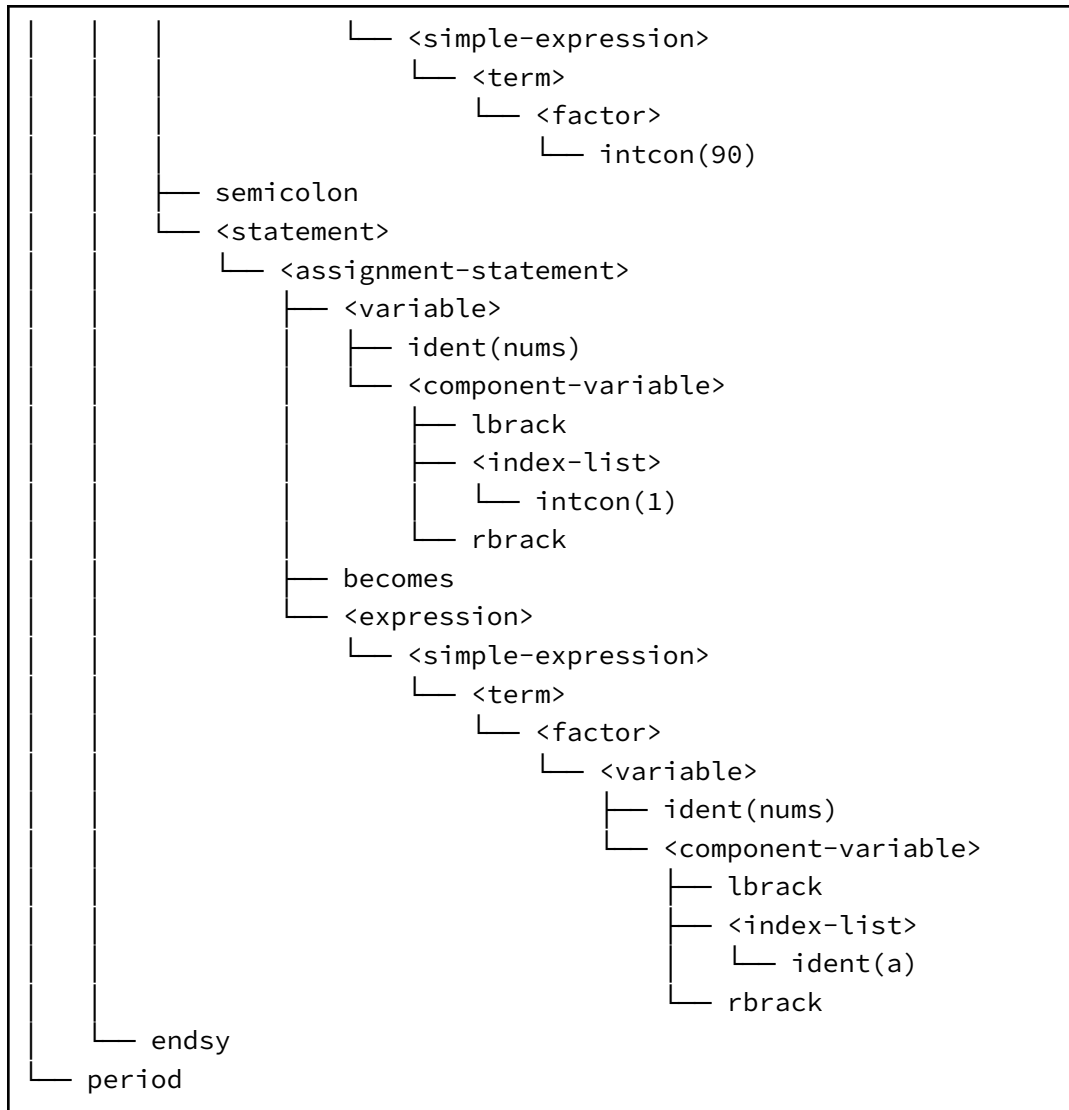
— colon
— <type>
  — ident(integer)
— semicolon
— <identifier-list>
  — ident(p)
— colon
— <type>
  — ident(Point)
— semicolon
— <identifier-list>
  — ident(s)
— colon
— <type>
  — ident(Student)
— semicolon
— <identifier-list>
  — ident(nums)
— colon
— <type>
  — <array-type>
    — arraysy
    — lbrack
    — <range>
      — <constant>
        — intcon(1)
      — period
      — period
      — <constant>
        — intcon(10)
    — rbrack
    — ofsy
    — <type>
      — ident(integer)
— semicolon
— <identifier-list>
  — ident(arr)
— colon
— <type>
  — <array-type>
    — arraysy
    — lbrack
    — ident(SmallInt)

```









3.2.7. Kasus Uji 7

- **Kasus:** Deklarasi tipe range menggunakan satu period (.) alih-alih dua period (..).
- **Tujuan:** Memverifikasi parser mendeteksi error ketika identifier nama program tidak ditemukan setelah keyword program.
- **Hasil:** **Lulus**

Kode Program:

```

program TestError1;
{ Error: parseRange - hanya satu period, seharusnya '..' }
type
    BadRange == 'a'..'z';
  
```

```
var
    x: integer;
begin
    x := 1
end.
```

Hasil Uji:

Syntax error at line 0, col 0: unexpected token 'charcon('z')', expected period

3.2.8. Kasus Uji 8

- **Kasus:** Deklarasi tipe array tidak menyertakan keyword 'of' sebelum tipe elemen.
- **Tujuan:** Memverifikasi parseArrayType mendeteksi error ketika keyword 'of' tidak ditemukan setelah rbrack.
- **Hasil:** **Lulus**

Kode Program:

```
program TestError2;
{ Error: parseArrayType - keyword 'of' hilang }
type
    BadArray == array[1..10] integer;
var
    x: integer;
begin
```

Hasil Uji:

Syntax error at line 4, col 34: unexpected token 'ident(integer)', expected ofsy

3.2.9. Kasus Uji 9

- **Kasus:** Deklarasi tipe enumerated tidak memiliki rparent (tanda kurung tutup) di akhir.
- **Tujuan:** Memverifikasi parseEnumerated mendeteksi error ketika rparent penutup tidak ditemukan.
- **Hasil:** **Lulus**

Kode Program:

```
program TestError3;
```

```
{ Error: parseEnumerated - rparent penutup hilang }
type
    BadEnum == (Red, Green, Blue;
var
    x: integer;
begin
    x := 1
end.
```

Hasil Uji:

Syntax error at line 4, col 34: unexpected token 'semicolon(;)', expected rparent

3.2.10. Kasus Uji 10

- **Kasus:** Lorem Ipsum
- **Tujuan:** Lorem Ipsum
- **Hasil:** Lulus

Kode Program:

```
program TestError4;
{ Error: parseVarDeclaration - colon antara identifier dan tipe
hilang }
var
    x, y integer;
begin
    x := 1
end.
```

Hasil Uji:

Syntax error at line 4, col 17: unexpected token 'ident(integer)', expected colon

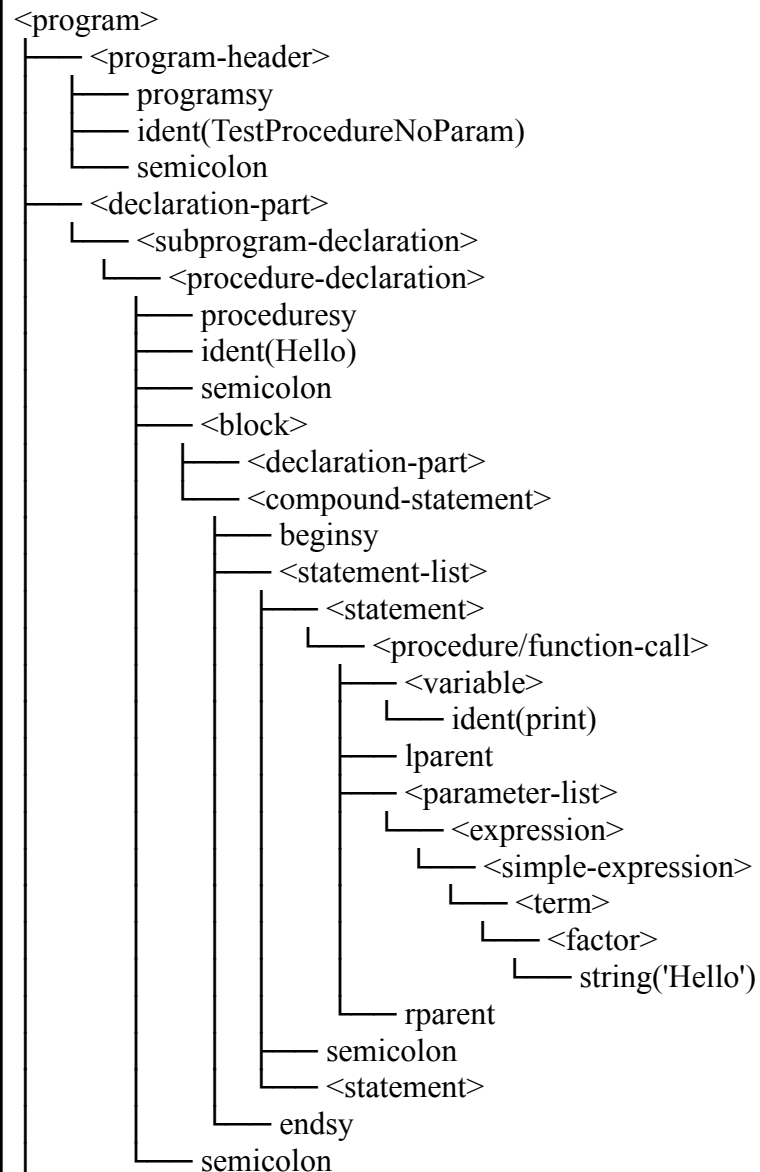
3.2.11. Kasus Uji 11

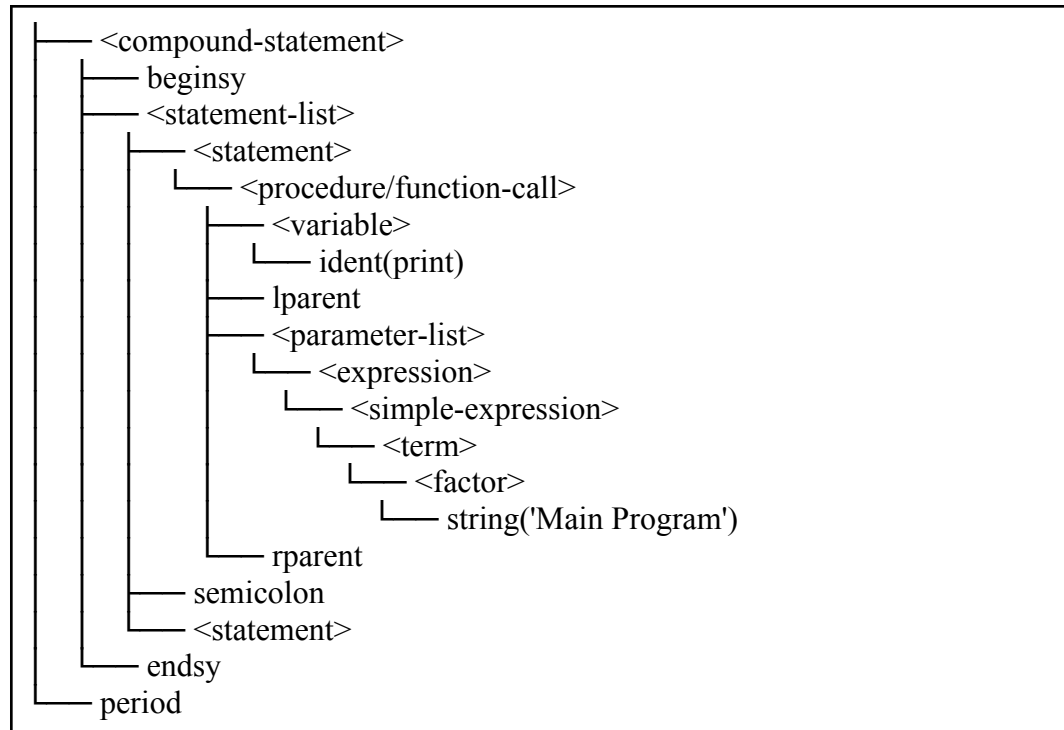
- **Kasus:** Program valid dengan deklarasi procedure tanpa parameter.
- **Tujuan:** Memverifikasi parser dapat memproses deklarasi subprogram berupa procedure tanpa formal parameter. Kasus ini menguji <subprogram-declaration>, <procedure-declaration>, dan <block> dengan struktur sederhana.
- **Hasil:** Lulus

Kode Program:

```
program TestProcedureNoParam;  
  
procedure Hello;  
begin  
    print('Hello');  
end;  
  
begin  
    print('Main Program');  
end.
```

Hasil Uji:





3.2.12. Kasus Uji 12

- **Kasus:** Program valid dengan procedure yang memiliki satu parameter.
- **Tujuan:** Memverifikasi parser dapat memproses formal parameter list pada procedure. Kasus ini menguji <formal-parameter-list> dan <parameter-group> dengan bentuk x: integer.
- **Hasil:** **Lulus**

Kode Program:

```

program TestProcedureOneParam;

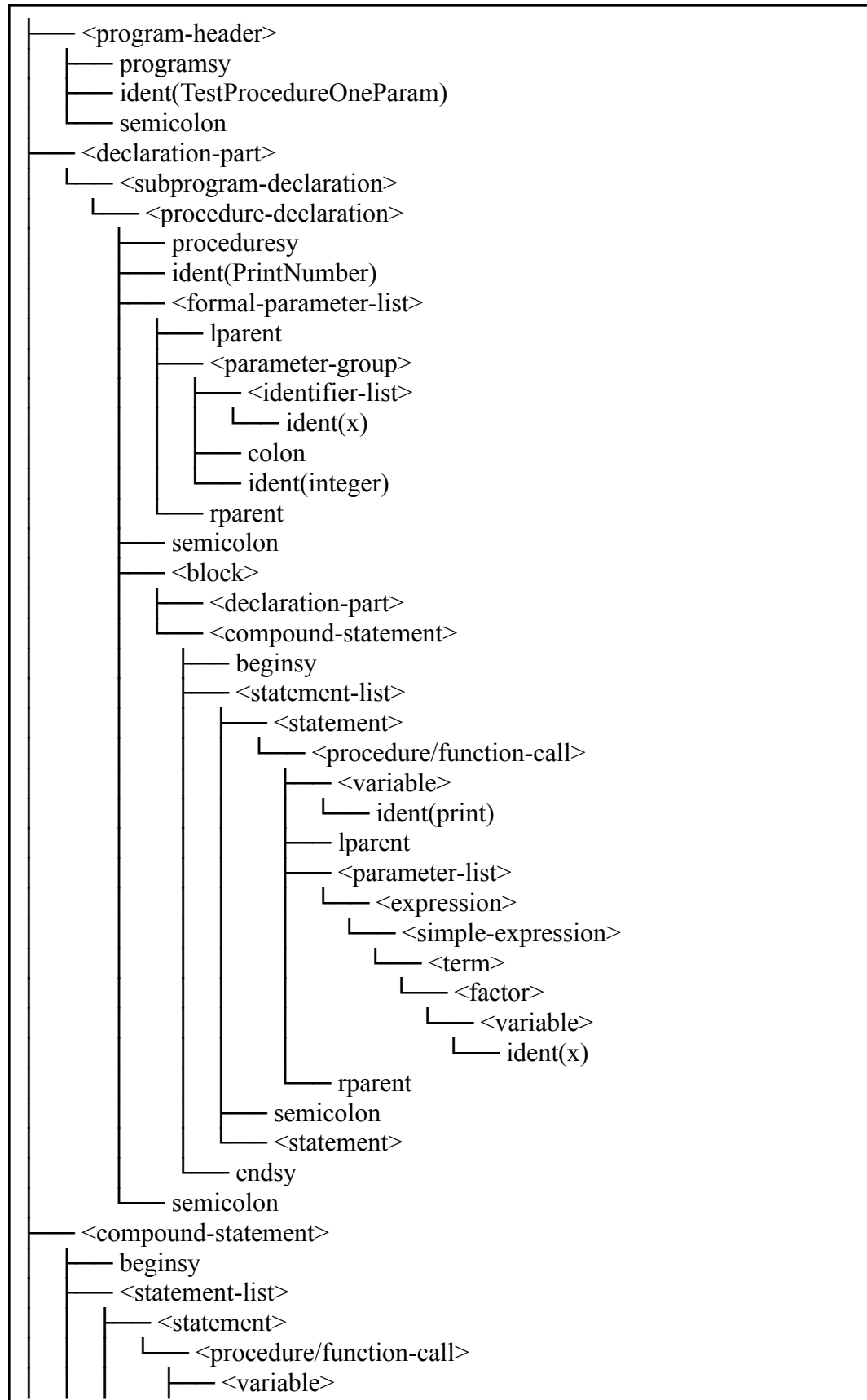
procedure PrintNumber(x: integer);
begin
  print(x);
end;

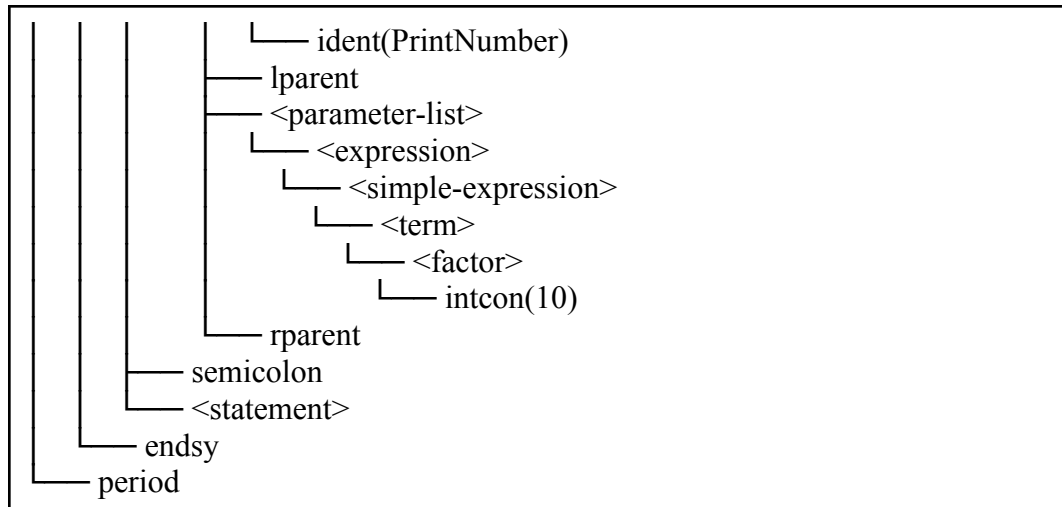
begin
  PrintNumber(10);
end.

```

Hasil Uji:

```
<program>
```





3.2.13. Kasus Uji 13

- **Kasus:** Program valid dengan procedure yang memiliki beberapa parameter group.
- **Tujuan:** Memverifikasi parser dapat memproses formal parameter list yang terdiri dari lebih dari satu parameter group dan dipisahkan oleh semicolon. Kasus ini juga menguji identifier-list pada parameter, seperti x, y: integer; name: string.
- **Hasil:** **Lulus**

Kode Program:

```

program TestProcedureMultiParam;

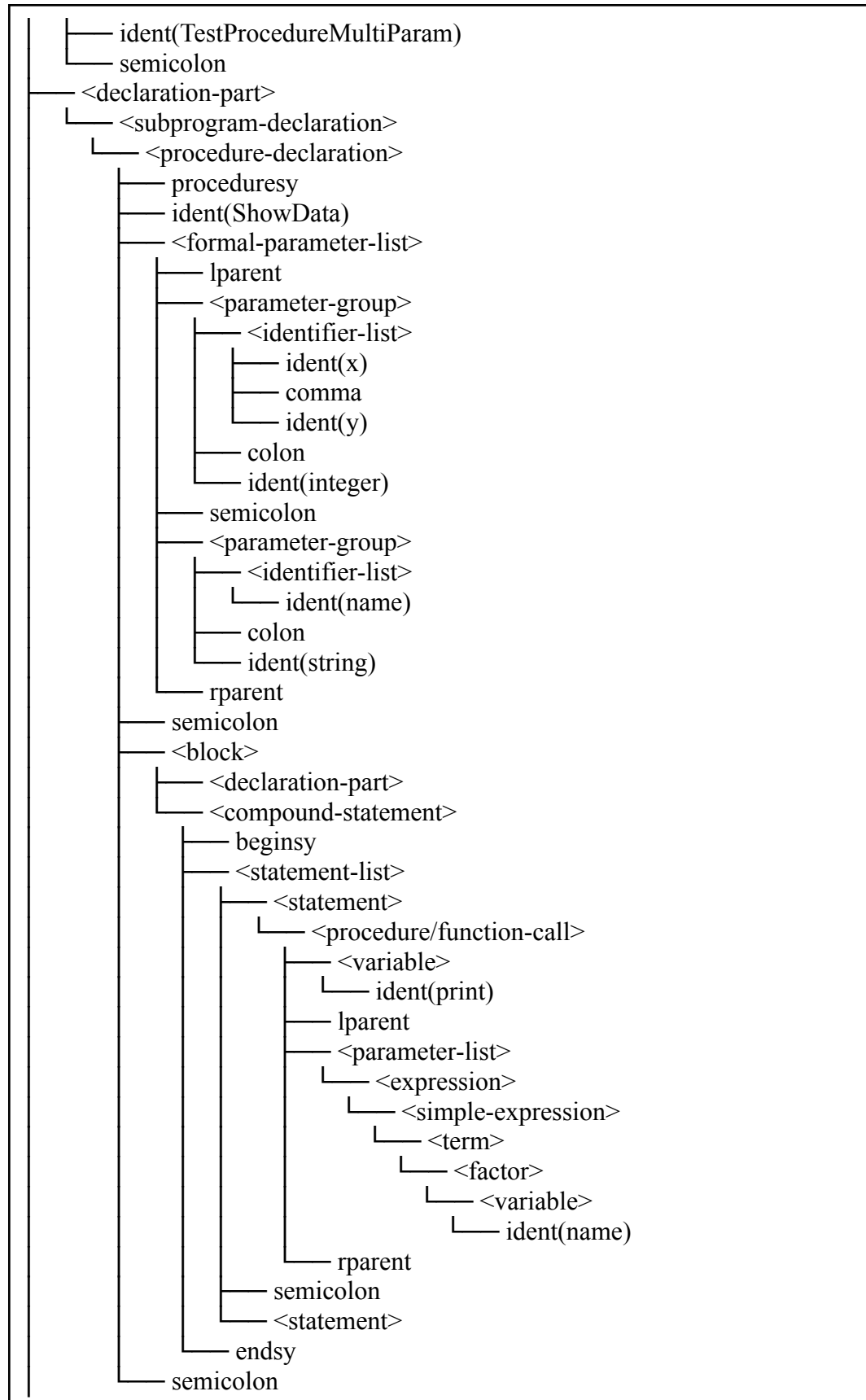
procedure ShowData(x, y: integer; name: string);
begin
    print(name);
end;

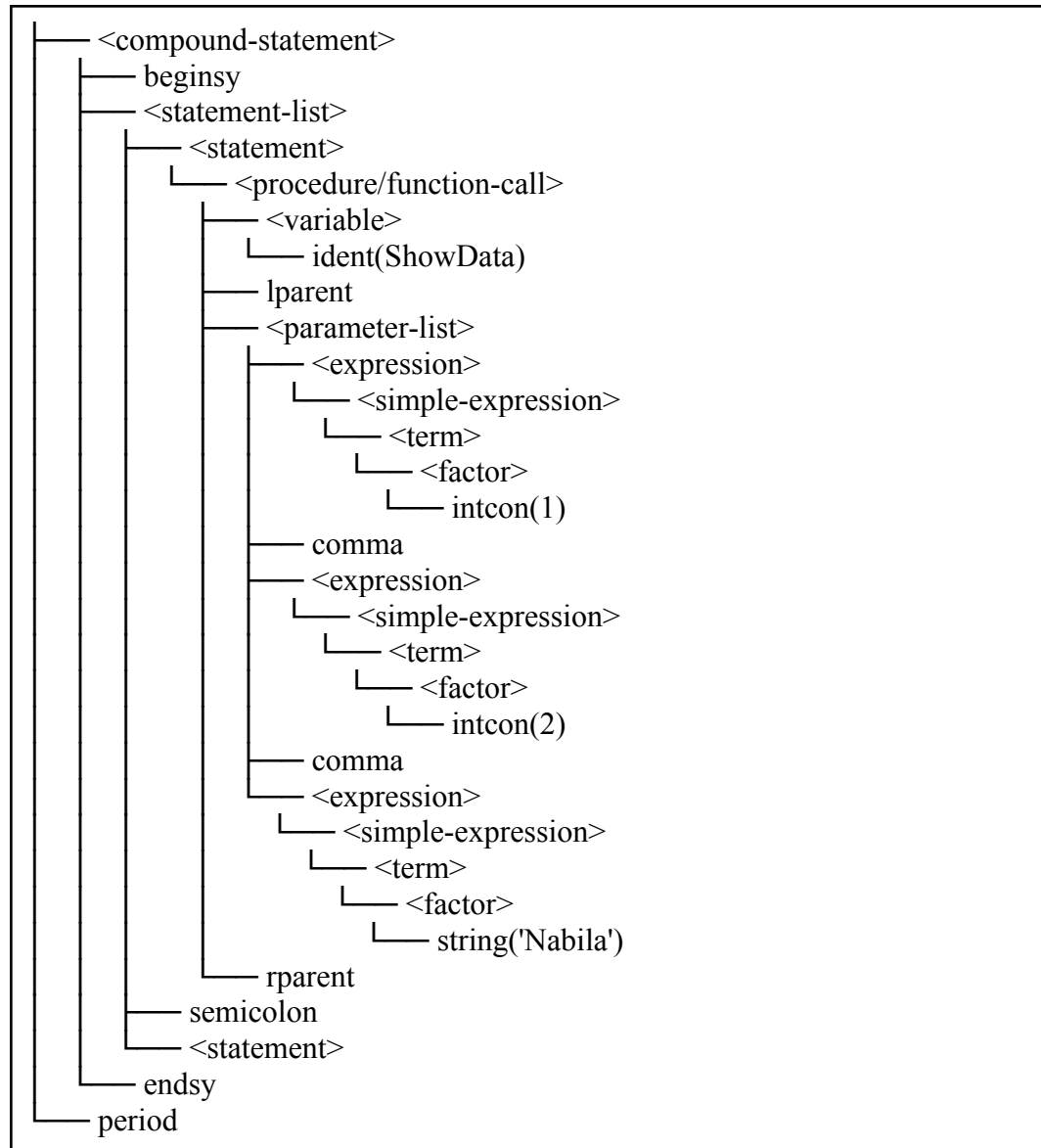
begin
    ShowData(1, 2, 'Nabila');
end.
  
```

Hasil Uji:

```

<program>
├── <program-header>
│   └── programsy
  
```



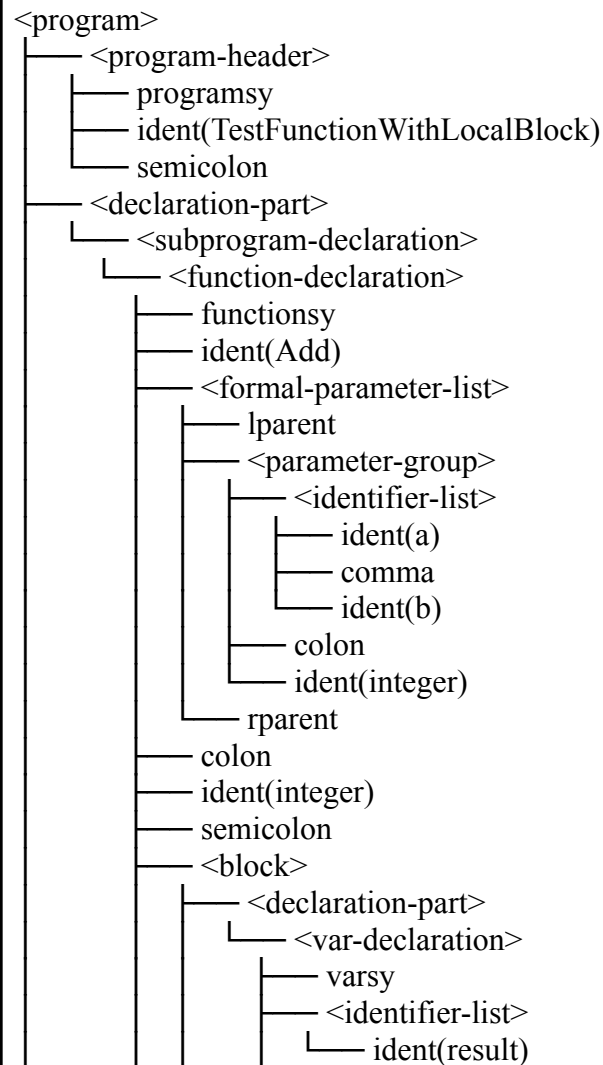
3.2.14. Kasus Uji 14

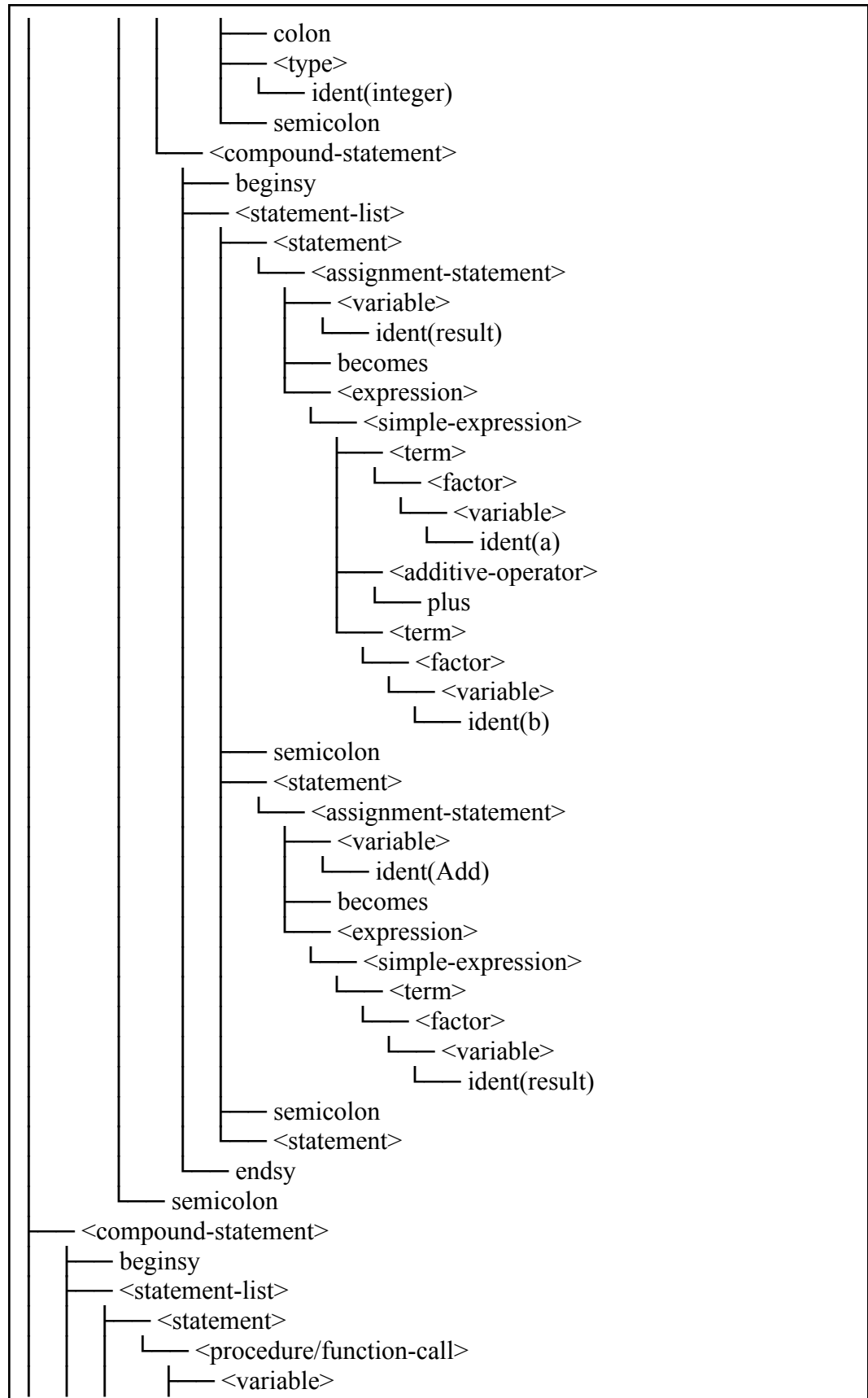
- **Kasus:** Program valid dengan deklarasi function, return type, dan deklarasi lokal di dalam block.
- **Tujuan:** Memverifikasi parser dapat memproses <function-declaration> yang memiliki parameter, colon, return type, dan block yang berisi declaration-part lokal. Kasus ini menguji apakah <block> dapat memproses deklarasi variabel lokal sebelum compound statement.
- **Hasil:** Lulus

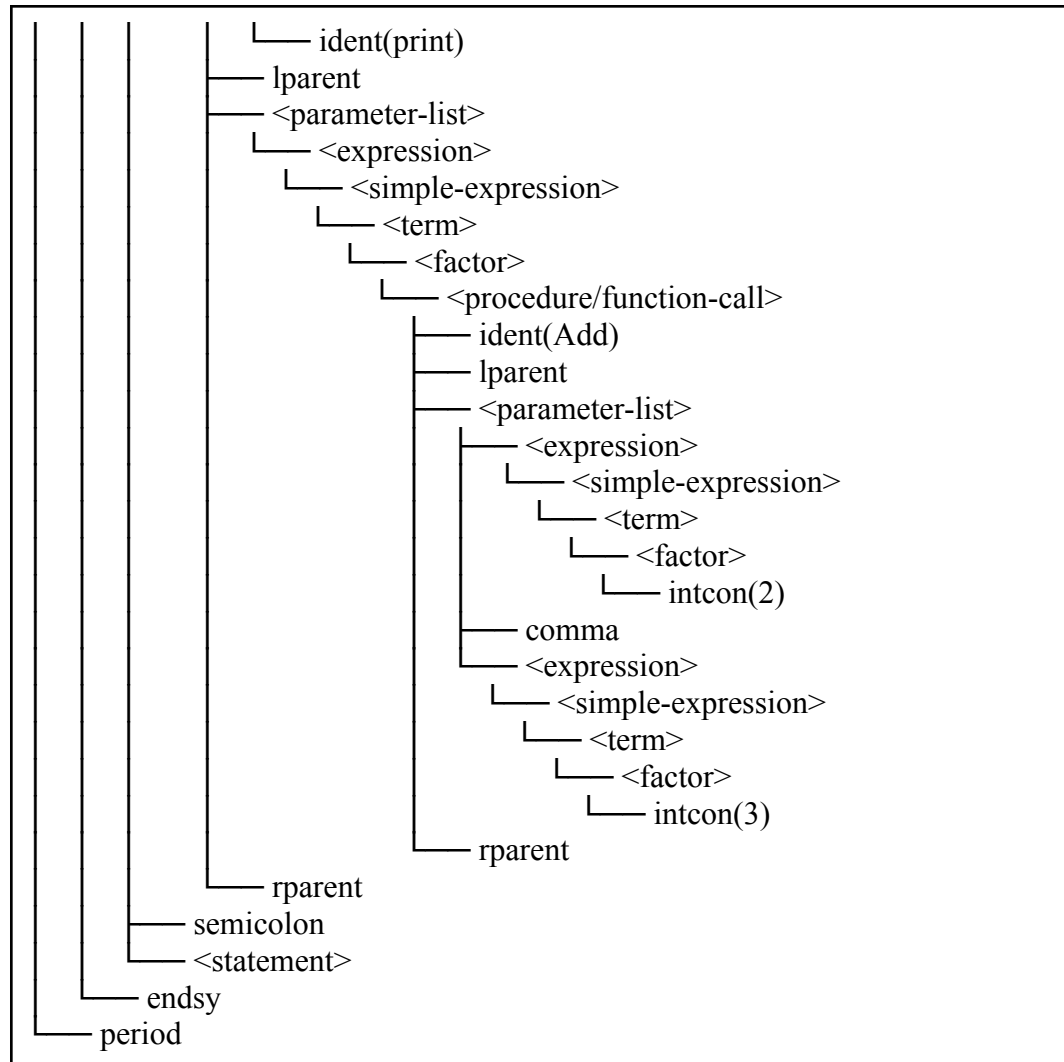
Kode Program:

```
program TestFunctionWithLocalBlock;  
  
function Add(a, b: integer): integer;  
var  
    result: integer;  
begin  
    result := a + b;  
    Add := result;  
end;  
  
begin  
    print(Add(2, 3));  
end.
```

Hasil Uji:







3.2.15. Kasus Uji 15

- **Kasus:** Program invalid dengan beberapa kesalahan pada deklarasi subprogram dan block.
- **Tujuan:** Memverifikasi parser dapat mendeteksi kesalahan pada bagian yang menjadi tanggung jawab Orang 3, yaitu parameter tanpa colon, function tanpa colon sebelum return type, dan block procedure yang tidak diawali dengan begin.
- **Hasil:** **Lulus**

Kode Program:

```
program TestInvalidSubprogram;
```

```

procedure PrintNumber(x integer);
begin
    print(x);
end;

function Add(a, b: integer) integer;
begin
    Add := a + b;
end;

procedure BrokenBlock;
    print('Hello');
end;

begin
    PrintNumber(10);
    print(Add(1, 2));
end.

```

Hasil Uji:

Syntax error at line 3, col 32: unexpected token 'ident(integer)', expected colon
 Syntax error at line 8, col 36: unexpected token 'ident(integer)', expected colon
 Syntax error at line 14, col 10: unexpected token 'ident(print)', expected beginsy

3.2.16. Kasus Uji 16

- **Kasus:** Compound statement dengan for loop, array indexing, assignment, dan procedure call
- **Tujuan:** Menguji `parseForStatement`, `parseVariable` + `parseComponentVariable` (array index), `parseAssignmentStatement`, dan `parseProcedureFunctionCall` secara bersamaan
- **Hasil:** **Lulus**

Kode Program:

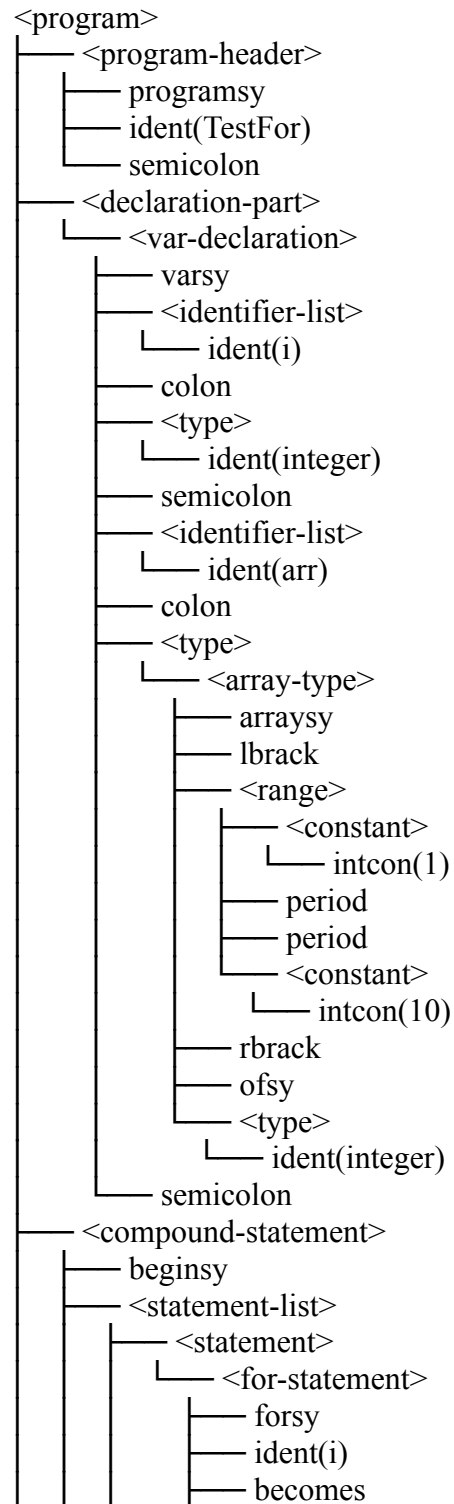
```

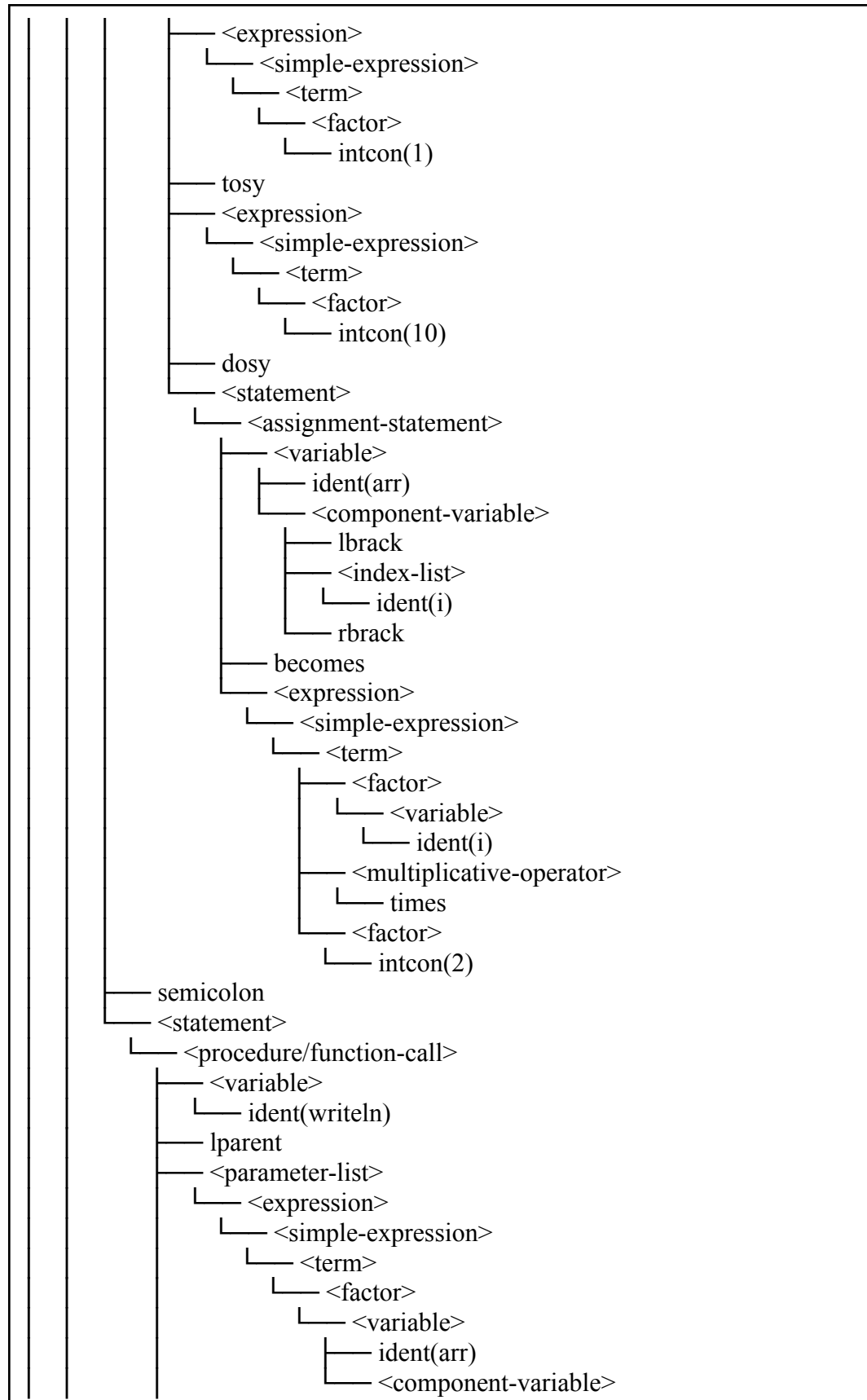
program TestFor;
var
    i : integer;
    arr : array[1..10] of integer;
begin
    for i := 1 to 10 do
        arr[i] := i * 2;
        writeln(arr[i])
    end
end.

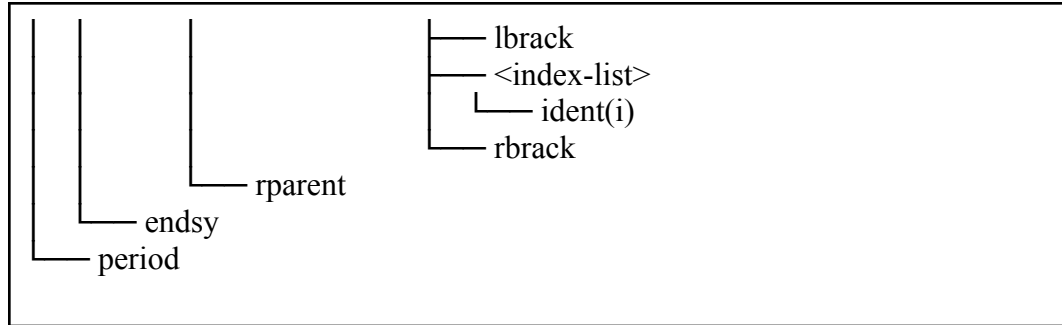
```

end.

Hasil Uji:







3.2.17. Kasus Uji 17

- **Kasus:** Case statement dengan multiple constant di satu case-block (1, 2:) dan case-block rekursif
- **Tujuan:** Menguji `parseCaseStatement` → `parseCaseBlock` rekursif dan parsing `constant + (comma + constant)*`
- **Hasil:** **Lulus**

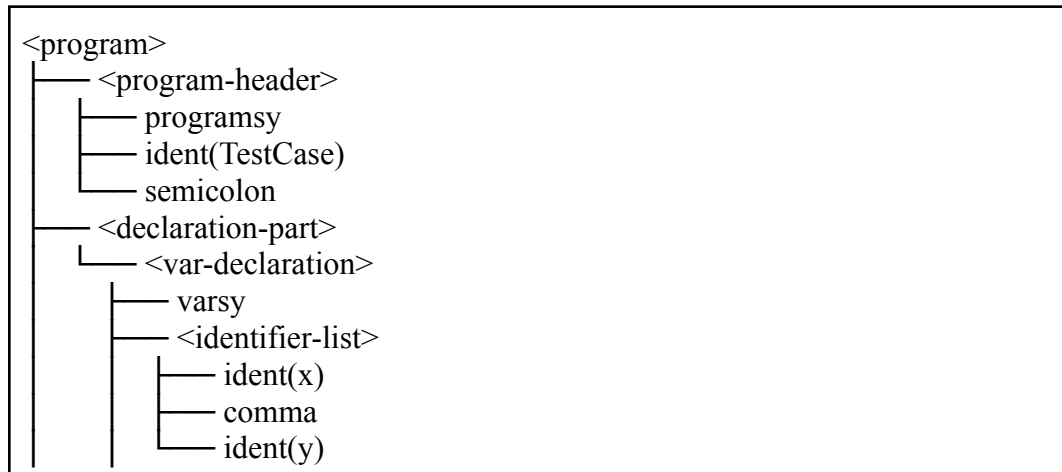
Kode Program:

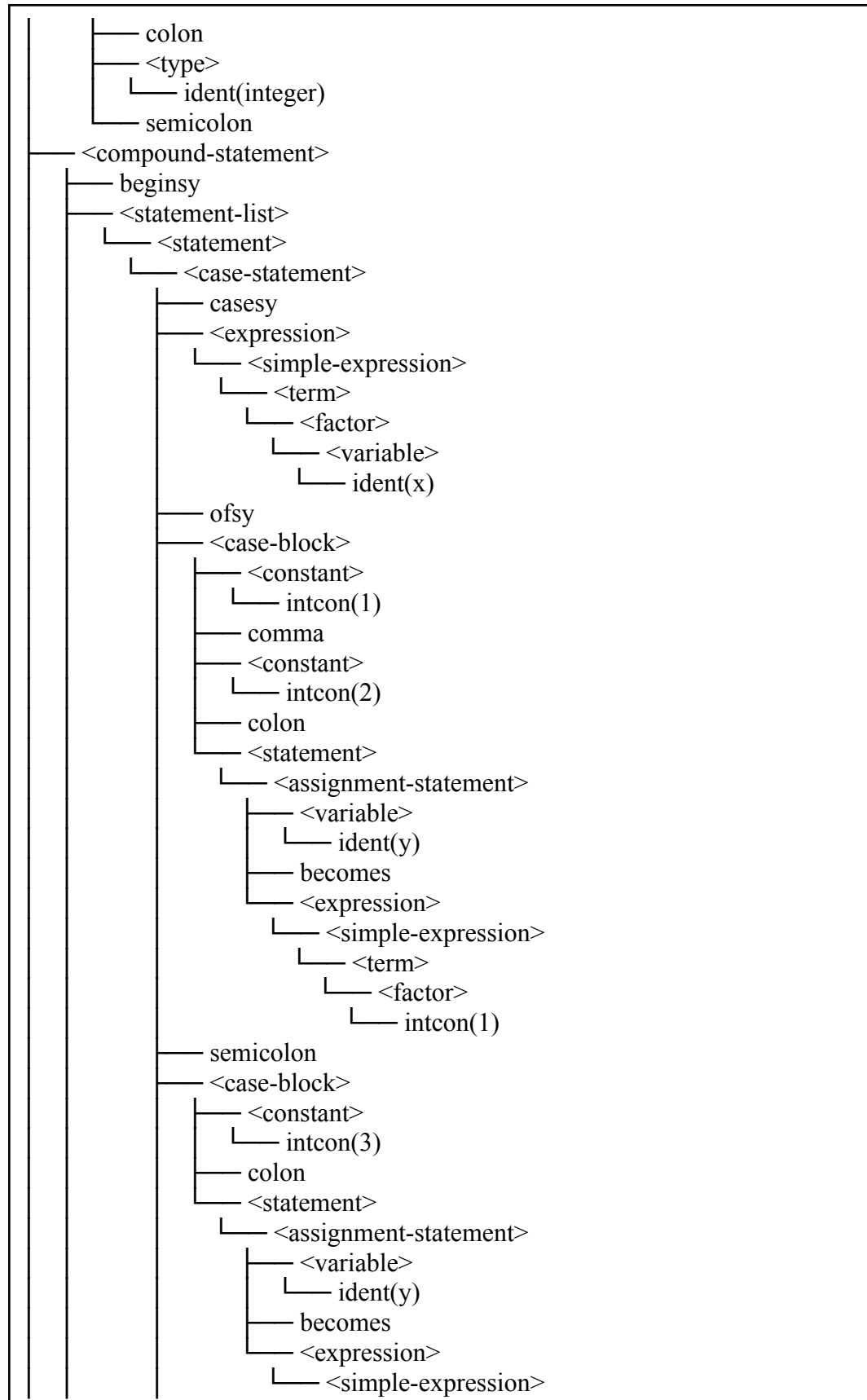
```

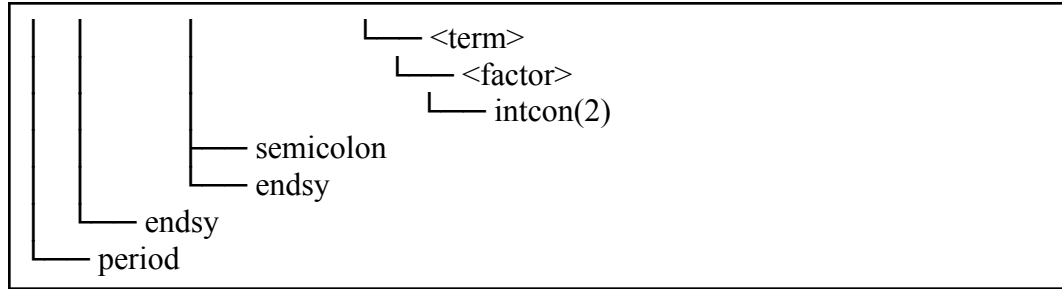
program TestCase;
var
  x, y : integer;
begin
  case x of
    1, 2: y := 1;
    3: y := 2;
  end
end.

```

Hasil Uji:







3.2.18. Kasus Uji 18

- **Kasus:** Repeat-until yang di dalamnya terdapat assignment dan if-else bertingkat
- **Tujuan:** parseRepeatStatement $\rightarrow$ parseStatementList $\rightarrow$ parseIfStatement dengan else branch, serta parseExpression pada kondisi until
- **Hasil:** **Lulus**

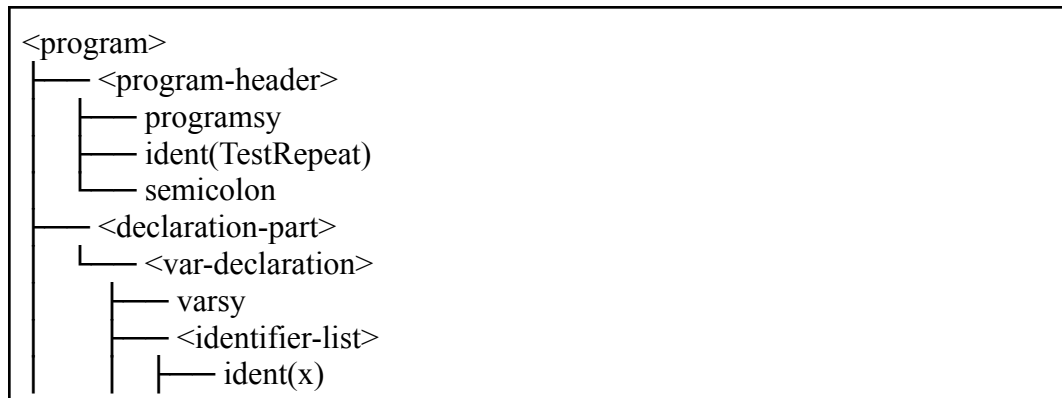
Kode Program:

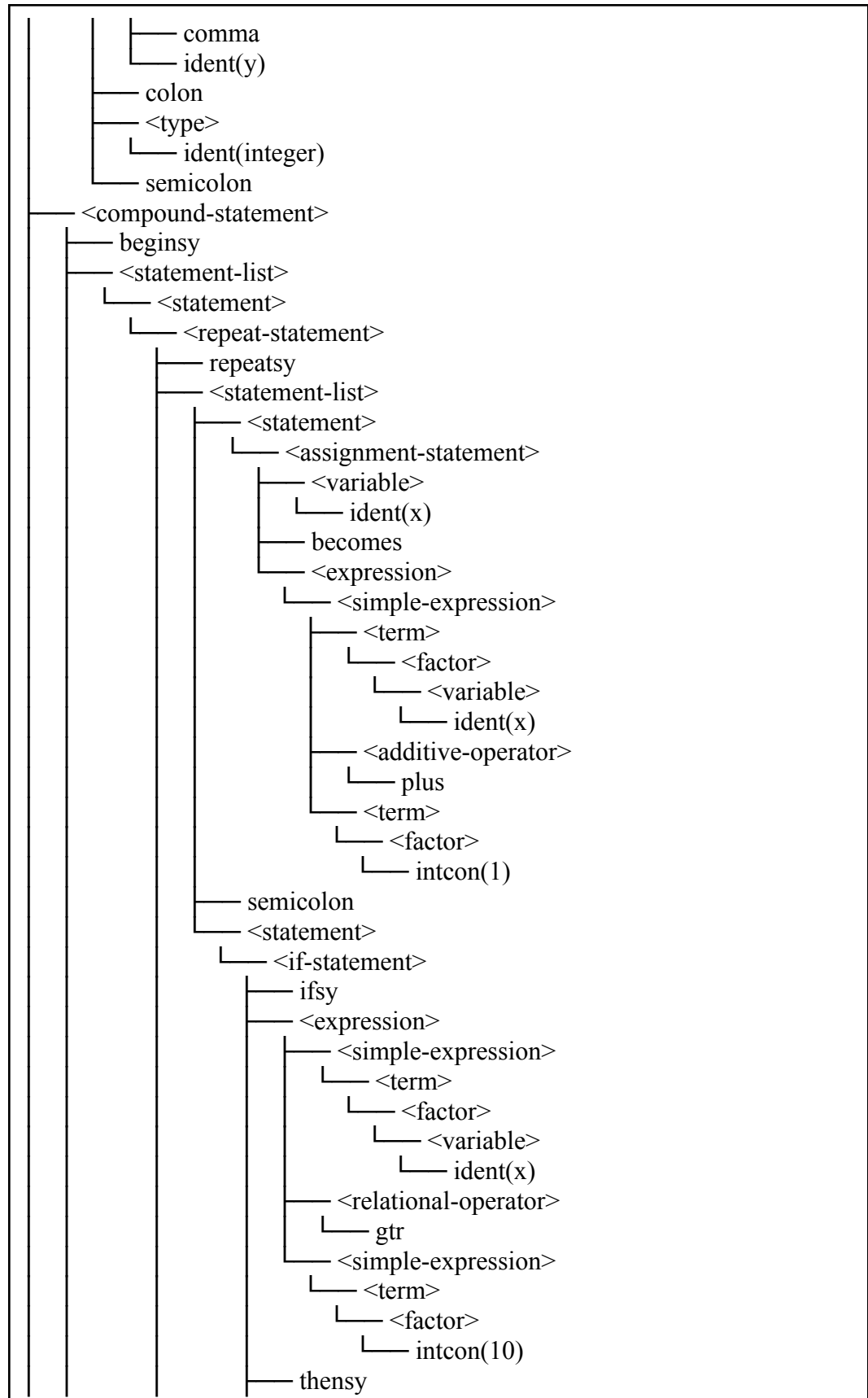
```

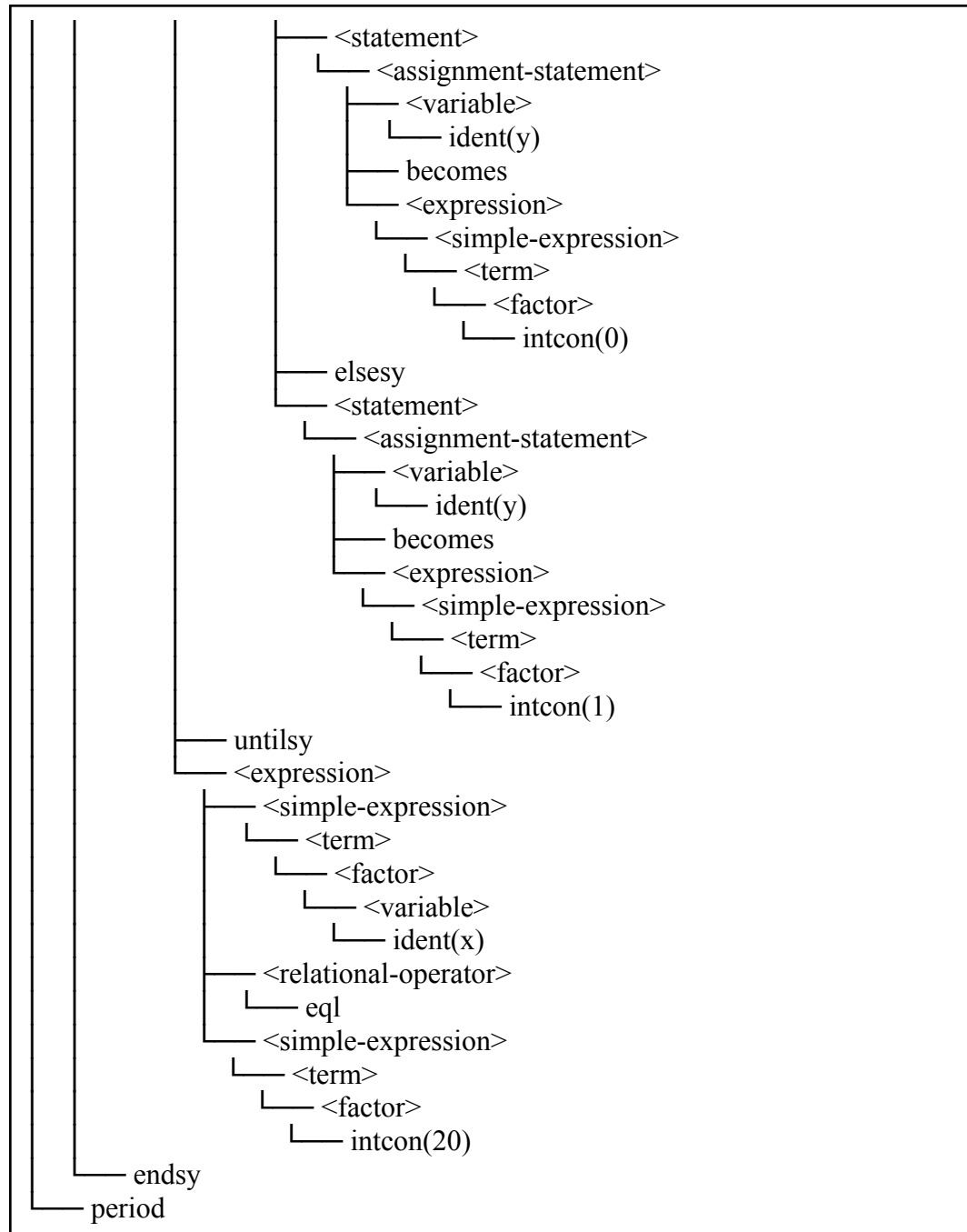
program TestRepeat;
var
  x, y : integer;
begin
  repeat
    x := x + 1;
    if x > 10 then
      y := 0
    else
      y := 1
    until x == 20
  end.

```

Hasil Uji:







3.2.19. Kasus Uji 19

- **Kasus:** While loop dengan assignment ke hasil function call yang parameternya kompleks (ekspresi dan array index)

- **Tujuan:** Menguji `parseWhileStatement`, `parseAssignmentStatement`, `parseProcedureFunctionCall` + `parseParameterList` dengan ekspresi bertingkat, dan `parseComponentVariable` untuk akses array
- **Hasil:** **Lulus**

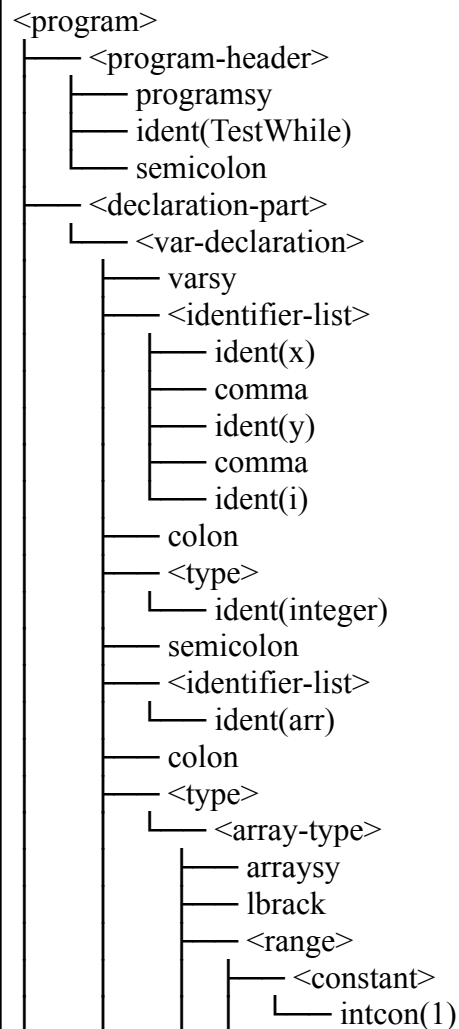
Kode Program:

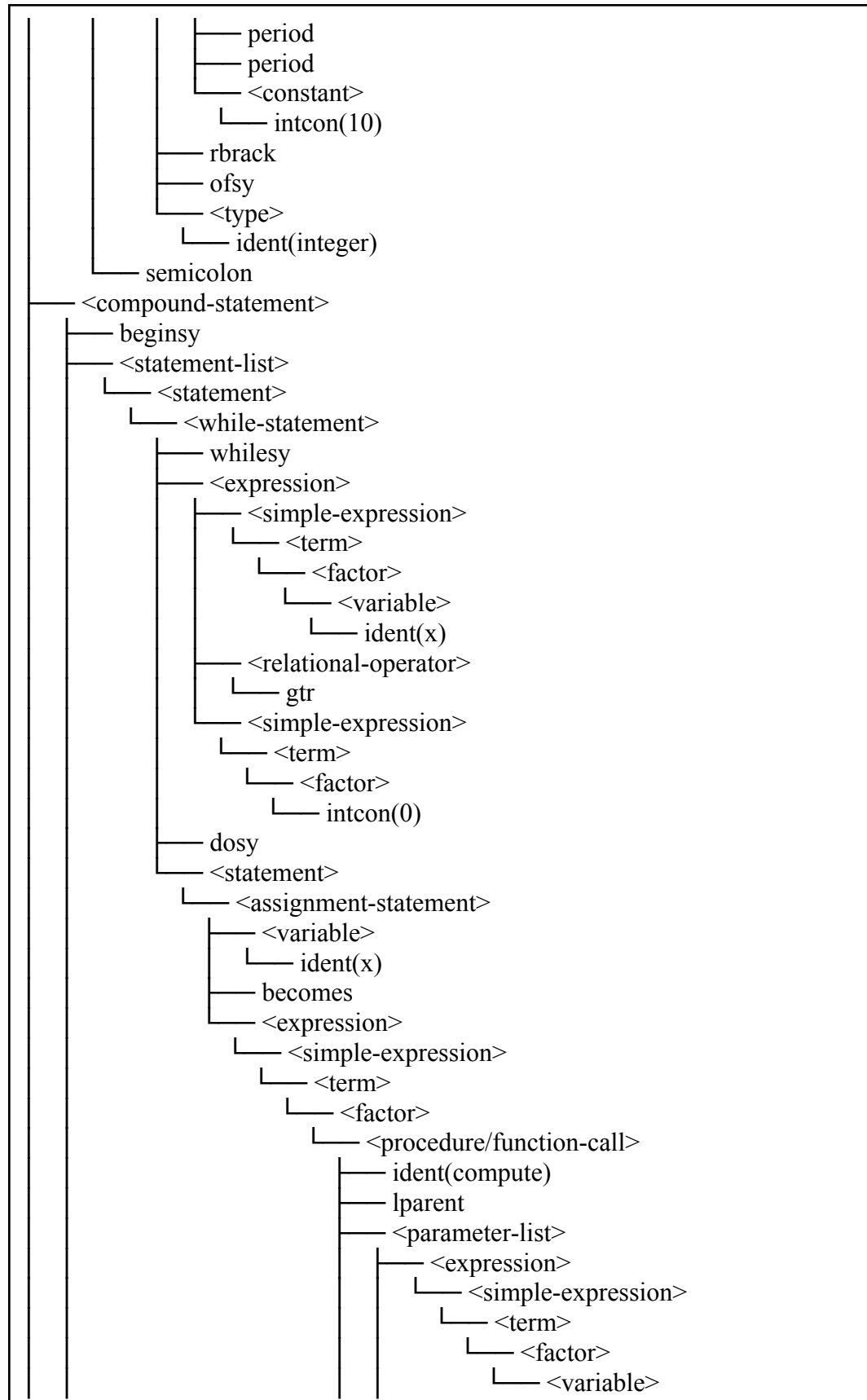
```

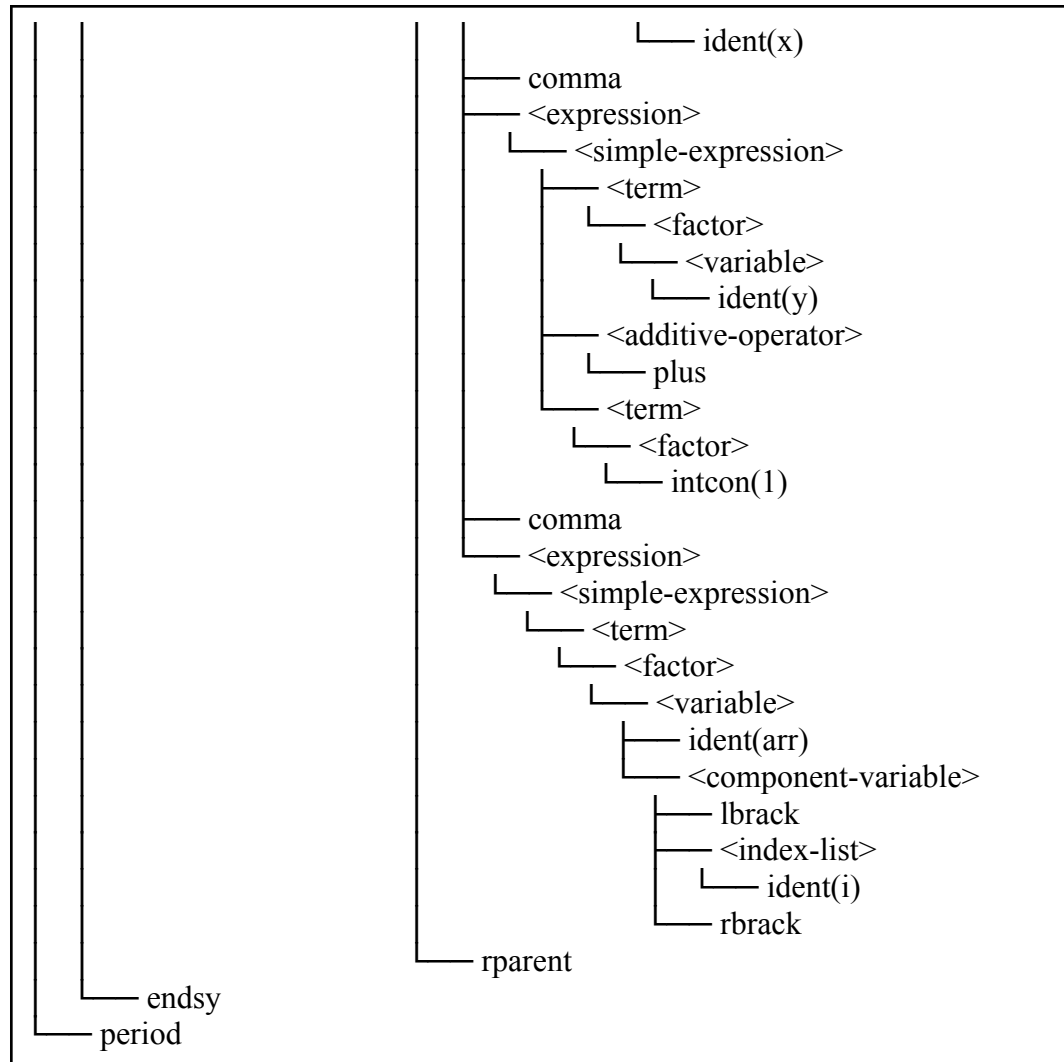
program TestWhile;
var
  x, y, i : integer;
  arr : array[1..10] of integer;
begin
  while x > 0 do
    x := compute(x, y + 1, arr[i])
  end.

```

Hasil Uji:







3.2.20. Kasus Uji 20

- **Kasus:** Case statement dengan semicolon yang hilang di antara case-block
- **Tujuan:** Menguji error recovery — setelah parseStatement selesai mem-parse `x := 1`, parser mengharapkan semicolon atau endsy tapi menemukan `intcon(2)`, sehingga `synchronize()` harus diaktifkan
- **Hasil:** **Lulus**

Kode Program:

```

program TestCaseMissingSemicolon;
var
  x, y : integer;
begin
  case y of

```



```
1: x := 1
2: x := 2
end
end.
```

Hasil Uji:

Syntax error at line 7, col 6: unexpected token 'intcon(2)', expected endsy
Syntax error at line 9, col 4: unexpected token 'endsy(end)', expected period
Syntax error at line 9, col 4: unexpected token 'endsy', expected end of program

3.2.21. Kasus Uji 21

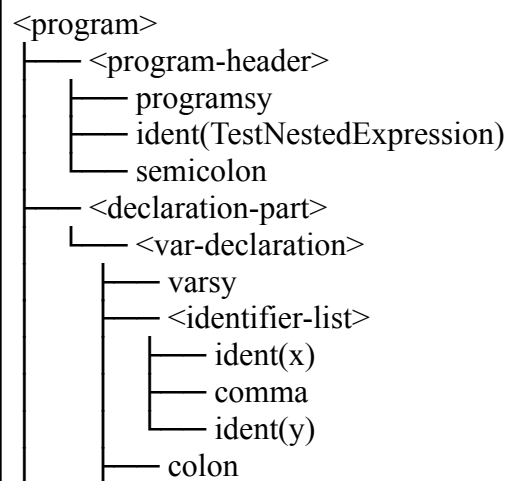
- **Kasus:** Case nested expression
- **Tujuan:** Menguji pemanggilan fungsi `parseExpression()` dan `parseSimpleExpression()` secara berurutan (nested)
- **Hasil:** **Lulus**

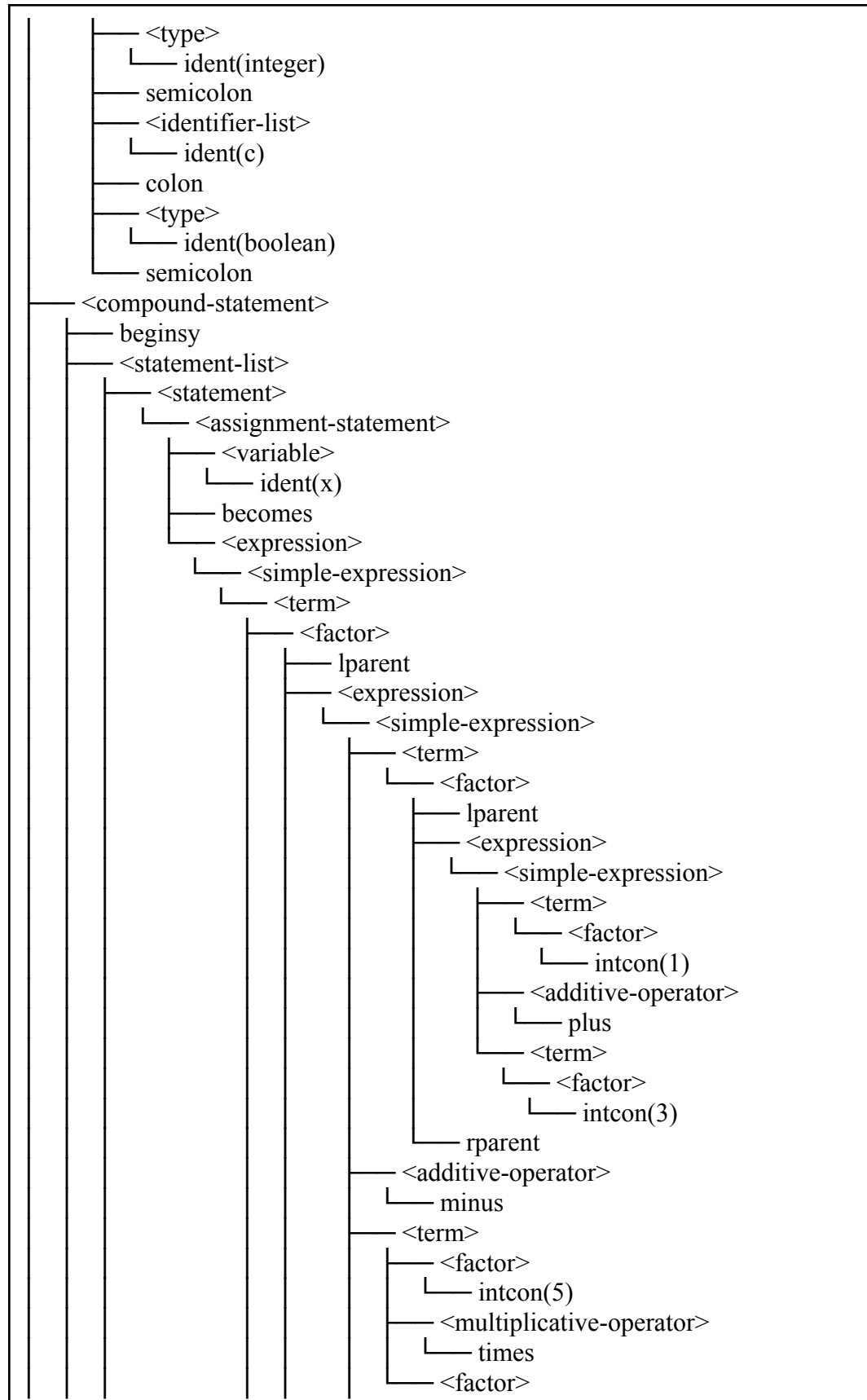
Kode Program:

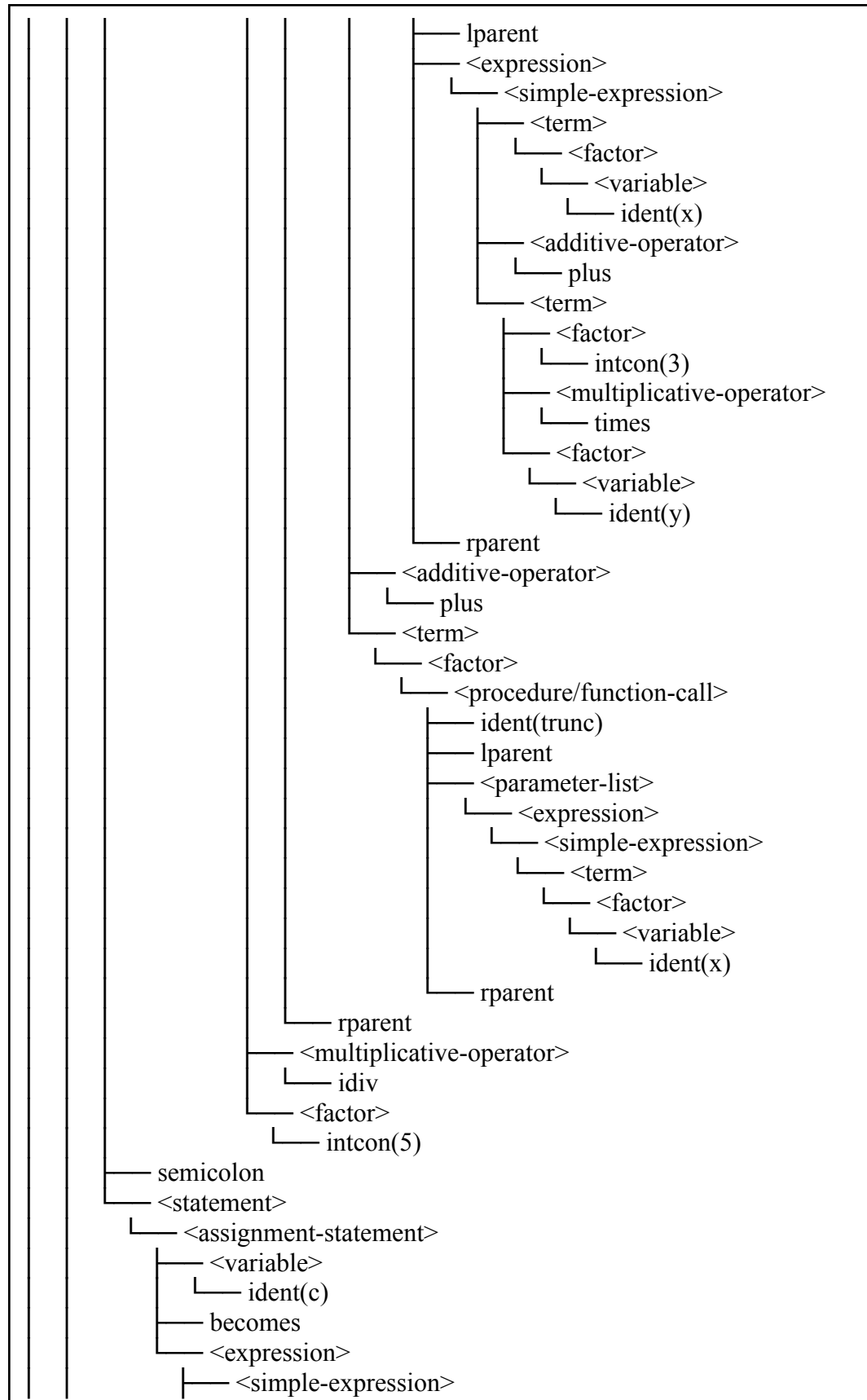
```
program TestNestedExpression;
var
  x, y : integer;
  c : boolean;

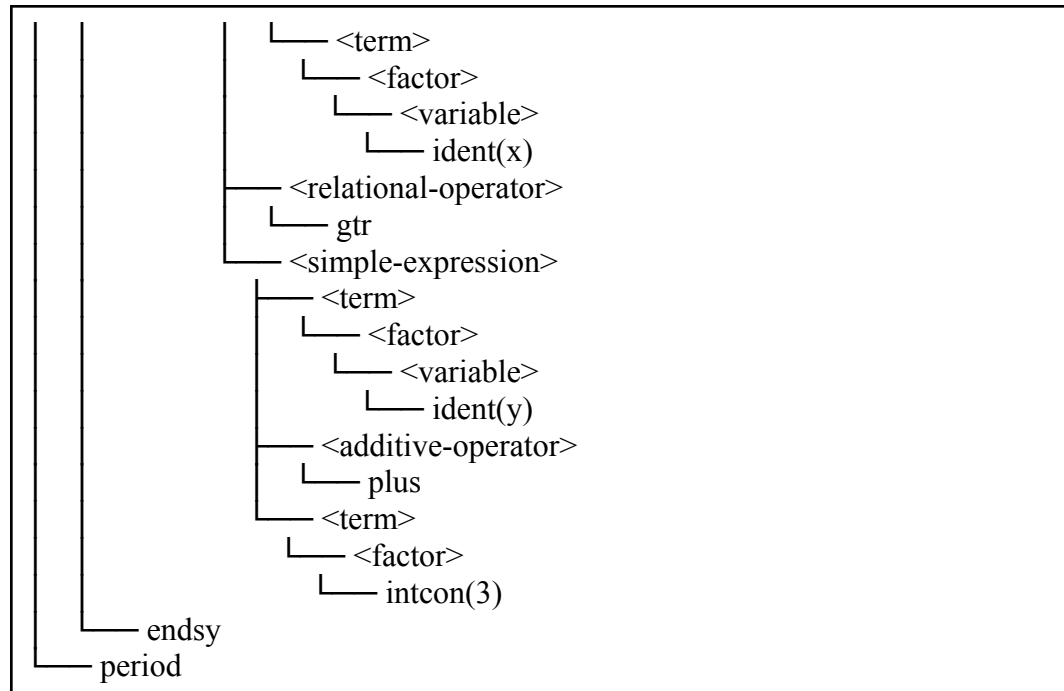
begin
  x := ((1 + 3) - 5 * (x + 3 * y) + trunc(x)) div 5;
  c := x > y + 3
end.
```

Hasil Uji:









3.2.22. Kasus Uji 22

- Kasus: Invalid expression
- Tujuan: Menguji pemanggilan fungsi `parseExpression()` dan `parseSimpleExpression()` yang tidak memenuhi syarat
- Hasil: **Lulus**

Kode Program:

```

program TestFalseExpression;
var
  x, y : integer;
  c : boolean;

begin
  x := ((1 + 3) - 5 > (x k 3 + y) + trunc(x)) div 5
  c := x > y + 3
end.

```

Hasil Uji:

Syntax error at line 7, col 27: unexpected token 'ident(k)', expected rparent
 Syntax error at line 8, col 4: unexpected token 'ident(c)', expected rparent

3.2.23. Kasus Uji 23

- Kasus: Invalid Expression (2)
- Tujuan: Menguji pemanggilan fungsi `parseExpression()`, `parseSimpleExpression()` yang tidak memenuhi syarat
- Hasil: **Lulus**

Kode Program:

```
program TestFalseOperator;  
var  
  x, y : integer;  
  c : boolean;  
  
begin  
  x := 3 > > 3;  
  c := x > y + 3  
end.
```

Hasil Uji:

```
Syntax error at line 7, col 13: unexpected token 'gtr(>)', expected ident  
Syntax error at line 8, col 4: unexpected token 'ident(c)', expected endsy  
Syntax error at line 9, col 4: unexpected token 'endsy(end)', expected period  
Syntax error at line 9, col 4: unexpected token 'endsy', expected end of program
```

BAB IV

KESIMPULAN DAN SARAN

4.1.Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang telah dilakukan pada Milestone 2, dapat ditarik beberapa kesimpulan sebagai berikut:

- Program Parser (syntax analyzer) untuk bahasa pemrograman Arion telah berhasil diimplementasikan menggunakan bahasa pemrograman C++. Implementasi ini dibangun secara mandiri tanpa menggunakan library atau tools pembantu otomatis, sesuai dengan spesifikasi yang diminta.
- Proses pengenalan struktur sintaks berhasil dimodelkan menggunakan algoritma Recursive Descent Parsing. Setiap nonterminal dalam grammar bahasa Arion dipetakan menjadi tepat satu fungsi rekursif, sehingga struktur kode parser mencerminkan secara langsung aturan produksi grammar yang telah dirancang.
- Parser mampu menerima aliran token hasil lexer dari Milestone 1 dan membangun parse tree yang merepresentasikan struktur hierarkis program Arion secara komprehensif, mencakup deklarasi, statement, ekspresi, hingga pemanggilan prosedur dan fungsi.
- Berdasarkan skenario pengujian yang telah dilakukan, program terbukti mampu menangani berbagai konstruksi bahasa Arion seperti assignment statement, control flow (if, while, repeat, for, case), akses array dan field record melalui component variable, serta pemanggilan prosedur dan fungsi dengan atau tanpa parameter, tanpa menyebabkan crash pada program.

4.2.Saran

Untuk pengembangan selanjutnya, terutama untuk milestone berikutnya, terdapat beberapa saran yang dapat diterapkan:

- Saat ini pesan error yang dihasilkan parser hanya menginformasikan token yang tidak terduga dan token yang diharapkan. Ke depannya, informasi error dapat dikembangkan dengan menambahkan posisi baris dan kolom terjadinya kesalahan pada source code agar proses debugging menjadi jauh lebih mudah bagi pengguna.

- Implementasi saat ini menggunakan mekanisme error handling yang langsung menghentikan proses parsing ketika menemukan kesalahan pertama. Untuk milestone selanjutnya, dapat dipertimbangkan penambahan mekanisme error recovery menggunakan synchronizing word seperti semicolon atau endsy, sehingga parser dapat melanjutkan analisis dan melaporkan lebih dari satu kesalahan dalam satu kali kompilasi.
- Struktur program yang sudah terpisah berdasarkan tanggung jawabnya (Parser.h, Parser.cpp, lexer.cpp, dll.) harus terus dijaga dan ditingkatkan pemisahannya. Modularitas ini sangat disarankan untuk mempermudah integrasi dengan tahapan kompilasi selanjutnya, terutama semantic analysis yang akan membutuhkan akses ke parse tree yang dihasilkan pada milestone ini.

DAFTAR PUSTAKA

- Cooper, K. D., & Torczon, L. (2012). Engineering a compiler (2nd ed.). Morgan Kaufmann.
- Spivak, R. (2015, December 15). Let's Build A Simple Interpreter. Part 7: Abstract Syntax Trees. Ruslan's Blog. <https://ruslanspivak.com/lsbasi-part7>
- Parsing Expressions · Crafting Interpreters. (n.d.). Craftinginterpreters.com. <https://craftinginterpreters.com/parsing-expressions.html>
- Pengenalan Context-Free Grammar (CFG) dan Derivasi. (2025, December 22). Naufarrel.dev. [https://if-notes.naufarrel.dev/02.-Catatan/IF2224-Teori-Bahasa-Formal-dan-Otomata/UA S/Pengenalan-Context-Free-Grammar-\(CFG\)-dan-Derivasi](https://if-notes.naufarrel.dev/02.-Catatan/IF2224-Teori-Bahasa-Formal-dan-Otomata/UA S/Pengenalan-Context-Free-Grammar-(CFG)-dan-Derivasi)
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2nd ed.). Pearson Education.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Pearson.
- Sipser, M. (2012). Introduction to the Theory of Computation (3rd ed.). Cengage Learning.

LAMPIRAN

Link Repository Github

Seluruh kode sumber program dapat diakses melalui repositori berikut.

<https://github.com/andraalanna/GTW-Tubes-IF2224-2026>

Pembagian Tugas

NIM	Nama	Tugas	Kontribusi
13524047	Muhammad Jordan Ferimeison	● Parser untuk ○ expression, ○ simple-expression, ○ term, ○ factor, ○ relational-operator, ○ additive-operator, ○ Multiplicative-operator ● Error handling ● Integrasi	20%
13524049	Arina Azka	● Landasan teori ● Parser untuk ○ subprogram-declaration, ○ procedure-declaration, ○ function-declaration, block, ○ formal-parameter-list, ○ Parameter-group ● Integrasi	20%
13524075	Hakam Avicena Mustain	● Parser untuk ○ compound-statement ○ Statement-list	20%

		○ index-list ○ variable ○ component-variable ○ statement ○ assignment-statement ○ if-statement ○ case-statement ○ case-block ○ while-statement ○ repeat-statement ○ for-statement ○ procedure-function-call ○ parameter-list ● Saran dan rekomendasi	
13524077	Yavie Azka Putra Araly	● Parser untuk ○ type-declaration ○ var-declaration ○ identifier-list ○ type ○ array-type ○ range ○ enumerated ○ record-type ○ field-list ○ field-part ○ ReadMe	20%
13524079	Angelina Andra Alana	● Landasan teori ● Revisi lexer Milestone 1 ● Parser untuk ○ program, ○ program-header,	Sy

		○ declaration-part, ○ const-declaration, ○ constant	
--	--	---	--